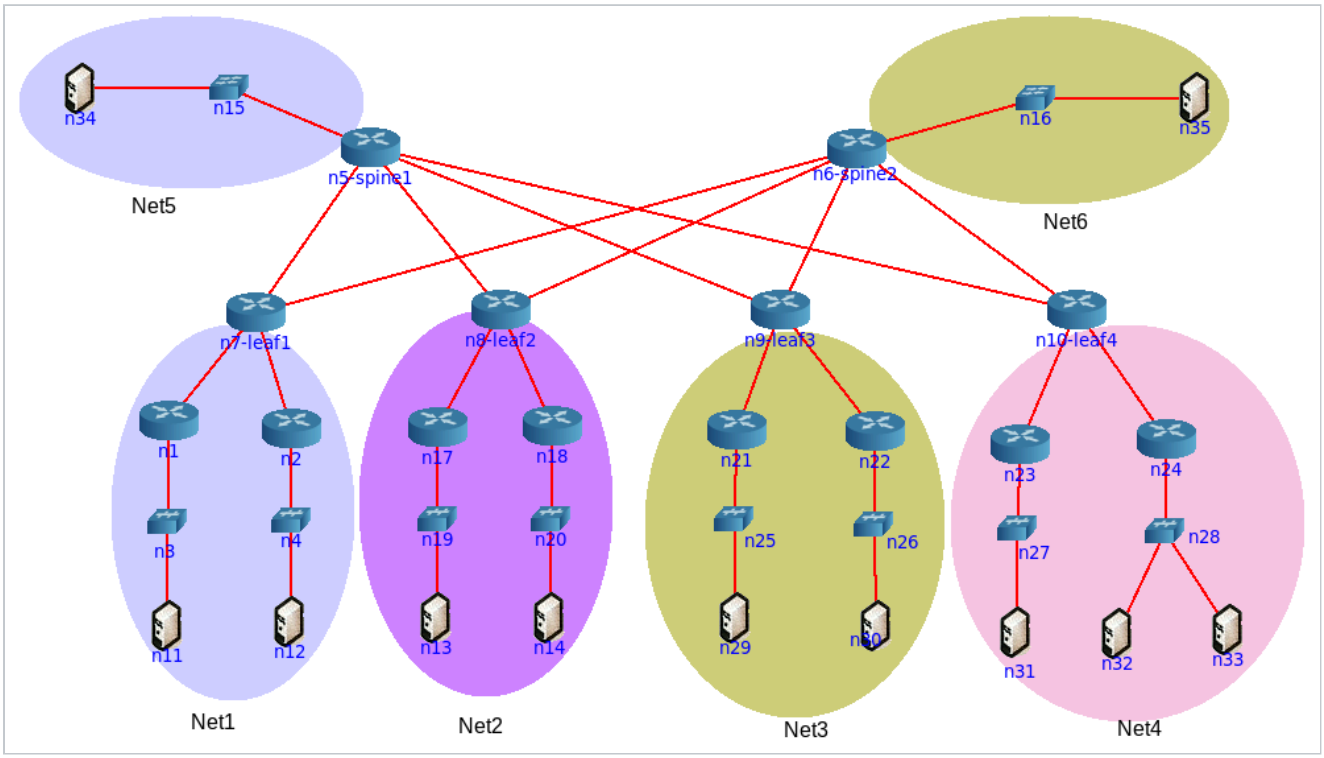
Trabajo Práctico Integrador — Redes de Datos II

Grupo 0X — 2026

Matías Lopez (02670/0) · Juan David Alvarez (03100/2) · Lautaro Nahuel Lopes (02978/9)



> *Figura 6 — topología asignada (enunciado, pág. 9)*

Datos

Bloque IPv4	Bloque IPv6
`16.0.0.0/16`	`1600::/32`

- Para `16.0.0.0/16`, el tamaño del bloque indica que hay **16 bits** (32 − 16) para hosts, es decir **65.536 direcciones** disponibles, 1 subred.
- Para `1600::/32`, el tamaño del bloque indica que hay **96 bits** (128 − 32) para hosts, 1 subred.

Topología: figura 6 del enunciado — arquitectura **spine-leaf** con 2 spines (`n5-spine1`, `n6-spine2`), 4 leafs (`n7-leaf1` … `n10-leaf4`), 8 routers de borde, 8 hubs, 2 switches y 11 PCs.

Nodo	Rol
n5, n6	spines
n7, n8, n9, n10	leafs
n1, n2, n17, n18, n21, n22, n23, n24	routers de LAN
n3, n4, n19, n20, n25, n26, n27, n28	hubs
n15, n16	switches
n11 – n14, n29 – n35	PCs

Direccionamiento IPv4

Análisis de requisitos

Se analizaron los requisitos de hosts de cada segmento para determinar la cantidad de direcciones necesarias. La fórmula aplicada es $2^n - 2 \geq \text{hosts}$, donde n es la cantidad de bits de host; se restan 2 por la dirección de red y la de broadcast.

Segmento	Hosts pedidos	Bits de host	Prefijo	Direcciones
Net5	500	$2^9 - 2 = 510 \rightarrow$ **9**	/23	512
Net1-A	200	$2^8 - 2 = 254 \rightarrow$ **8**	/24	256
Net1-B	200	$2^8 - 2 = 254 \rightarrow$ **8**	/24	256
Net2-A	100	$2^7 - 2 = 126 \rightarrow$ **7**	/25	128
Net3-A	100	$2^7 - 2 = 126 \rightarrow$ **7**	/25	128
Net3-B	60	$2^6 - 2 = 62 \rightarrow$ **6**	/26	64
Net2-B	30	$2^5 - 2 = 30 \rightarrow$ **5**	/27	32
Net4-A	20	$2^5 - 2 = 30 \rightarrow$ **5**	/27	32
Net4-B	20	$2^5 - 2 = 30 \rightarrow$ **5**	/27	32
Net6	10	$2^4 - 2 = 14 \rightarrow$ **4**	/28	16

Subtotal LANs: 1456 direcciones.

El enunciado indica considerar los enlaces punto a punto `/30` o `/31`. Se contabilizaron **16 enlaces punto a punto**:

- 8 entre cada router de LAN y su leaf (2 por cada Net1 a Net4)
- 8 entre cada leaf y los dos spines (4 leafs × 2 spines)

> **Net5 y Net6 no tienen enlace punto a punto.** El enunciado los menciona únicamente como "*una red de 500 hosts*" y "*una red de 10 hosts*", sin la aclaración "*y las punto a punto con leaf*" que sí aparece en Net1 a Net4. En la topología, **n15 y n16 son switches**, por lo que la LAN de Net5 queda directamente conectada a una interfaz de spine1 y la de Net6 a una de spine2.

Se optó por `/30` en lugar de `/31` por claridad: cada enlace conserva su dirección de red y de broadcast explícitas, lo que facilita la lectura de las tablas de ruteo. El costo es de 2 direcciones por enlace (32 en total), despreciable frente al tamaño del bloque asignado.

Subtotal enlaces: 16 × 4 = 64 direcciones. Total: 1520 de 65.536 direcciones disponibles (2,3 % del bloque).

Asignación VLSM

Se aplicó VLSM sobre `16.0.0.0/16` comenzando por el segmento que requiere más direcciones y continuando en orden decreciente, de modo que cada bloque quede alineado a un múltiplo de su propio tamaño.

- El primer segmento es **Net5**, que requiere 512 direcciones (`/23`). Se le asigna `16.0.0.0/23`, que abarca de `16.0.0.0` a `16.0.1.255`.
- Siguen **Net1-A** y **Net1-B**, de 256 direcciones cada una (`/24`): que abarcan de `16.0.2.0/24` a `16.0.2.255/24` y de `16.0.3.0/24` a `16.0.3.255/24`.
- Siguen **Net2-A** y **Net3-A**, de 128 direcciones (`/25`). Se subnetea `16.0.4.0/24` en dos `/25`: `16.0.4.0/25` y `16.0.4.128/25`.
- Sigue **Net3-B**, de 64 direcciones (`/26`). Se subnetea `16.0.5.0/24`: `16.0.5.0/26` queda asignada.
- Siguen **Net2-B**, **Net4-A** y **Net4-B**, de 32 direcciones (`/27`): `16.0.5.64/27`, `16.0.5.96/27` y `16.0.5.128/27`.
- Sigue **Net6**, de 16 direcciones (`/28`): `16.0.5.160/28`.
- Finalmente, los **16 enlaces punto a punto** se agrupan en `16.0.6.0/26`, subneteado en 16 redes `/30` consecutivas de `16.0.6.0` a `16.0.6.60`.

Queda libre `16.0.5.176` – `16.0.5.255` y todo `16.0.6.64` – `16.0.255.255`.

Tabla de direccionamiento

Red	Segmento	Prefijo	Dirección de red	Primera	Última asignable	Broadcast
Net5	spine1 ↔ n15 ↔ n34	/23	`16.0.0.0`	16.0.0.1	16.0.1.254	`16.0.1.255`
Net1-A	n1 ↔ n3 ↔ n11	/24	`16.0.2.0`	16.0.2.1	16.0.2.254	`16.0.2.255`
Net1-B	n2 ↔ n4 ↔ n12	/24	`16.0.3.0`	16.0.3.1	16.0.3.254	`16.0.3.255`
Net2-A	n17 ↔ n19 ↔ n13	/25	`16.0.4.0`	16.0.4.1	16.0.4.126	`16.0.4.127`
Net3-A	n21 ↔ n25 ↔ n29	/25	`16.0.4.128`	16.0.4.129	16.0.4.254	`16.0.4.255`
Net3-B	n22 ↔ n26 ↔ n30	/26	`16.0.5.0`	16.0.5.1	16.0.5.62	`16.0.5.63`
Net2-B	n18 ↔ n20 ↔ n14	/27	`16.0.5.64`	16.0.5.65	16.0.5.94	`16.0.5.95`
Net4-A	n23 ↔ n27 ↔ n31	/27	`16.0.5.96`	16.0.5.97	16.0.5.126	`16.0.5.127`
Net4-B	n24 ↔ n28 ↔ n32, n33	/27	`16.0.5.128`	16.0.5.129	16.0.5.158	`16.0.5.159`
Net6	spine2 ↔ n16 ↔ n35	/28	`16.0.5.160`	16.0.5.161	16.0.5.174	`16.0.5.175`

Enlaces punto a punto — todos `/30`, máscara `255.255.255.252`

Red	Segmento	Extremo A	Extremo B
`16.0.6.0/30`	n1 ↔ leaf1	n1 → `.1`	leaf1 → `.2`
`16.0.6.4/30`	n2 ↔ leaf1	n2 → `.5`	leaf1 → `.6`
`16.0.6.8/30`	n17 ↔ leaf2	n17 → `.9`	leaf2 → `.10`
`16.0.6.12/30`	n18 ↔ leaf2	n18 → `.13`	leaf2 → `.14`
`16.0.6.16/30`	n21 ↔ leaf3	n21 → `.17`	leaf3 → `.18`
`16.0.6.20/30`	n22 ↔ leaf3	n22 → `.21`	leaf3 → `.22`
`16.0.6.24/30`	n23 ↔ leaf4	n23 → `.25`	leaf4 → `.26`
`16.0.6.28/30`	n24 ↔ leaf4	n24 → `.29`	leaf4 → `.30`
`16.0.6.32/30`	leaf1 ↔ spine1	leaf1 → `.33`	spine1 → `.34`
`16.0.6.36/30`	leaf1 ↔ spine2	leaf1 → `.37`	spine2 → `.38`
`16.0.6.40/30`	leaf2 ↔ spine1	leaf2 → `.41`	spine1 → `.42`
`16.0.6.44/30`	leaf2 ↔ spine2	leaf2 → `.45`	spine2 → `.46`
`16.0.6.48/30`	leaf3 ↔ spine1	leaf3 → `.49`	spine1 → `.50`
`16.0.6.52/30`	leaf3 ↔ spine2	leaf3 → `.53`	spine2 → `.54`
`16.0.6.56/30`	leaf4 ↔ spine1	leaf4 → `.57`	spine1 → `.58`
`16.0.6.60/30`	leaf4 ↔ spine2	leaf4 → `.61`	spine2 → `.62`

Tabla de interfaces

> Los nombres `ethN` están tomados del archivo `topologia.imn`, no asumidos: CORE los asigna según el orden en que se crearon los enlaces. **No son simétricos entre nodos equivalentes** — en leaf1 el enlace a spine1 es `eth2`, mientras que en leaf2, leaf3 y leaf4 es `eth3`.

Nodo	Interfaz	Segmento	IPv4	IPv6
n1	eth0	n1–leaf1	`16.0.6.1/30`	–
	eth1	LAN Net1-A	`16.0.2.1/24`	–
n2	eth0	n2–leaf1	`16.0.6.5/30`	–
	eth1	LAN Net1-B	`16.0.3.1/24`	–
n17	eth0	LAN Net2-A	`16.0.4.1/25`	–
	eth1	n17–leaf2	`16.0.6.9/30`	–
n18	eth0	LAN Net2-B	`16.0.5.65/27`	–
	eth1	n18–leaf2	`16.0.6.13/30`	–
n21	eth0	LAN Net3-A	`16.0.4.129/25`	–
	eth1	n21–leaf3	`16.0.6.17/30`	–
n22	eth0	LAN Net3-B	`16.0.5.1/26`	–
	eth1	n22–leaf3	`16.0.6.21/30`	–
n23	eth0	LAN Net4-A	`16.0.5.97/27`	–
	eth1	n23–leaf4	`16.0.6.25/30`	–
n24	eth0	LAN Net4-B	`16.0.5.129/27`	–
	eth1	n24–leaf4	`16.0.6.29/30`	–
leaf1 (n7)	eth0	n1–leaf1	`16.0.6.2/30`	–
	eth1	n2–leaf1	`16.0.6.6/30`	–
	eth2	leaf1–spine1	`16.0.6.33/30`	–
	eth3	leaf1–spine2	`16.0.6.37/30`	–
leaf2 (n8)	eth0	n17–leaf2	`16.0.6.10/30`	–
	eth1	n18–leaf2	`16.0.6.14/30`	–
	eth2	leaf2–**spine2**	`16.0.6.45/30`	–
	eth3	leaf2–**spine1**	`16.0.6.41/30`	–
leaf3 (n9)	eth0	n21–leaf3	`16.0.6.18/30`	–
	eth1	n22–leaf3	`16.0.6.22/30`	–
	eth2	leaf3–**spine2**	`16.0.6.53/30`	–
	eth3	leaf3–**spine1**	`16.0.6.49/30`	–
leaf4 (n10)	eth0	n23–leaf4	`16.0.6.26/30`	–
	eth1	n24–leaf4	`16.0.6.30/30`	–
	eth2	leaf4–**spine2**	`16.0.6.61/30`	–
	eth3	leaf4–**spine1**	`16.0.6.57/30`	–
spine1 (n5)	eth0	leaf1–spine1	`16.0.6.34/30`	–
	eth1	**LAN Net5**	`16.0.0.1/23`	`1600:0:0:5::1/64`

Nodo	Interfaz	Segmento	IPv4	IPv6
	eth2	leaf2–spine1	`16.0.6.42/30`	–
	eth3	leaf3–spine1	`16.0.6.50/30`	–
	eth4	leaf4–spine1	`16.0.6.58/30`	–
	tun6	túnel 6in4	–	`1600:0:0:ffff::1/64`
spine2 (n6)	eth0	leaf4–spine2	`16.0.6.62/30`	–
	eth1	leaf3–spine2	`16.0.6.54/30`	–
	eth2	**LAN Net6**	`16.0.5.161/28`	`1600:0:0:6::1/64`
	eth3	leaf2–spine2	`16.0.6.46/30`	–
	eth4	leaf1–spine2	`16.0.6.38/30`	–
	tun6	túnel 6in4	–	`1600:0:0:ffff::2/64`
n11	eth0	LAN Net1-A	`16.0.2.10/24`	–
n12	eth0	LAN Net1-B	`16.0.3.10/24`	–
n13	eth0	LAN Net2-A	`16.0.4.10/25`	–
n14	eth0	LAN Net2-B	`16.0.5.70/27`	–
n29	eth0	LAN Net3-A	`16.0.4.140/25`	–
n30	eth0	LAN Net3-B	`16.0.5.10/26`	–
n31	eth0	LAN Net4-A	`16.0.5.100/27`	–
n32	eth0	LAN Net4-B	`16.0.5.131/27`	–
n33	eth0	LAN Net4-B	`16.0.5.130/27`	–
n34	eth0	LAN Net5	`16.0.0.10/23`	`1600:0:0:5::10/64`
n35	eth0	LAN Net6	`16.0.5.170/28`	`1600:0:0:6::10/64`

Los hubs (n3, n4, n19, n20, n25–n28) y los switches (n15, n16) no llevan dirección IP: operan en capa 1 y capa 2 respectivamente.

Inciso A — Configurar Net5 y Net6 con IPv6

Se asigna el bloque `1600::/32`. Cada LAN lleva prefijo **/64**, que es el tamaño estándar en IPv6 independientemente de la cantidad de hosts, ya que los últimos 64 bits están reservados al *Interface ID* y no se subnetean.

Red	Prefijo	Nodo	Dirección
LAN Net5	`1600:0:0:5::/64`	spine1 eth1	`1600:0:0:5::1/64`
		n34	`1600:0:0:5::10/64`
LAN Net6	`1600:0:0:6::/64`	spine2 eth2	`1600:0:0:6::1/64`
		n35	`1600:0:0:6::10/64`
Túnel 6in4	`1600:0:0:ffff::/64`	spine1 tun6	`1600:0:0:ffff::1/64`
		spine2 tun6	`1600:0:0:ffff::2/64`

Mecanismo de transición: túnel 6in4

Para dar conectividad IPv6 entre Net5 y Net6 se optó por un **túnel 6in4** (modo `sit`, protocolo IP 41) en lugar de configurar IPv6 de forma nativa en todo el camino.

Justificación: Net5 cuelga de spine1 y Net6 de spine2. Un despliegue nativo exigiría configurar direcciones IPv6, rutas, forwarding y reglas de filtrado en los dos spines y en el leaf de tránsito. Con el túnel, los nodos intermedios solo ven datagramas IPv4 ordinarios y **la configuración IPv6 queda confinada a los dos extremos**. Es además uno de los mecanismos de transición previstos por la RFC 4213, lo que lo hace preferible a un despliegue parcial ad-hoc.

Como extremos del túnel se usan las interfaces IPv4 de cada spine hacia leaf1: `16.0.6.34` (spine1) y `16.0.6.38` (spine2). El túnel transita por leaf1.

```
# spine1
ip -6 addr add 1600:0:0:5::1/64 dev eth1
ip tunnel add tun6 mode sit remote 16.0.6.38 local 16.0.6.34 ttl 64
ip link set tun6 up
ip -6 addr add 1600:0:0:ffff::1/64 dev tun6
ip -6 route add 1600:0:0:6::/64 dev tun6

# spine2
ip -6 addr add 1600:0:0:6::1/64 dev eth2
ip tunnel add tun6 mode sit remote 16.0.6.34 local 16.0.6.38 ttl 64
ip link set tun6 up
ip -6 addr add 1600:0:0:ffff::2/64 dev tun6
ip -6 route add 1600:0:0:5::/64 dev tun6
```

En los nodos de CORE el forwarding IPv6 viene deshabilitado y la política por defecto de `ip6tables` es `DROP`, por lo que en ambos spines fue necesario además:

```
sysctl -w net.ipv6.conf.all.forwarding=1
ip6tables -P FORWARD ACCEPT
```

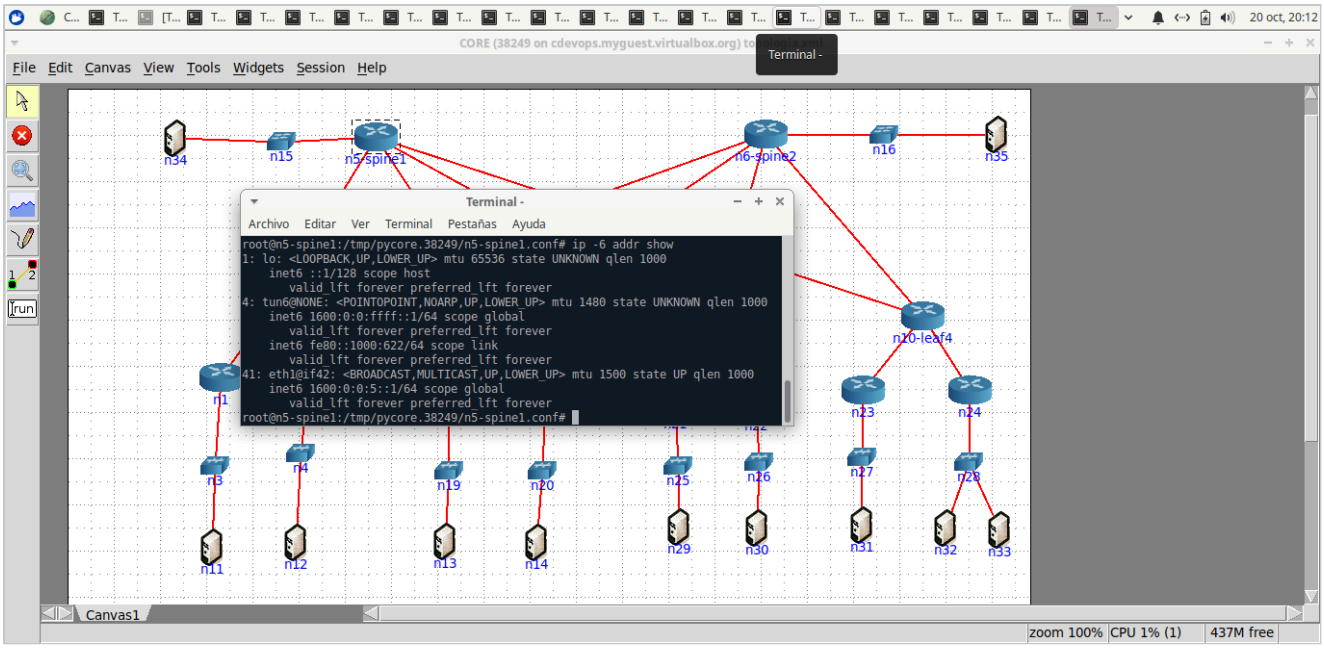


Figura A.1 — `ip -6 addr show` en spine1: LAN Net5 (`1600:0:0:5::1`) y extremo del túnel (`1600:0:0:ffff::1` , MTU 1480)

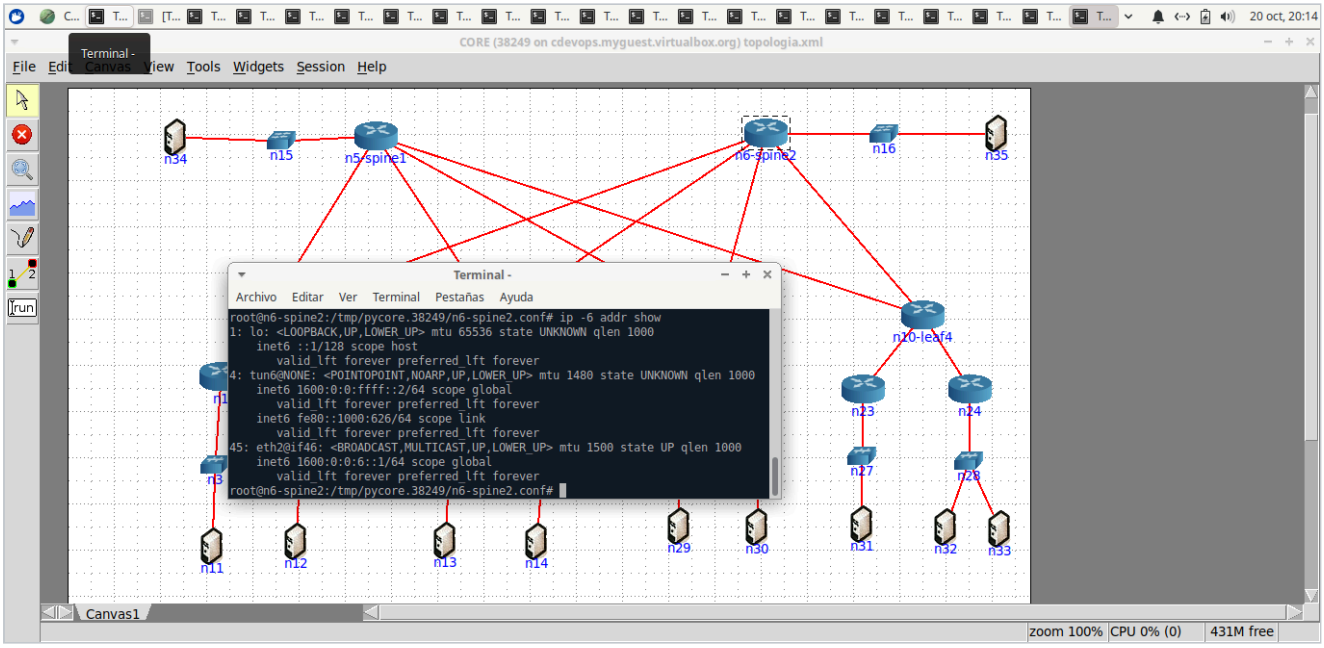


Figura A.2 — `ip -6 addr show` en spine2: LAN Net6 (`1600:0:0:6::1`) y extremo del túnel (`1600:0:0:ffff::2`)

Inciso B — Ruteo estático IPv4 e IPv6

Criterio adoptado

La topología spine-leaf tiene **caminos redundantes**: cada leaf se conecta a los dos spines. Con ruteo estático esto habilita bucles si se configuran rutas por defecto enfrentadas. El criterio aplicado para evitarlos es:

1. **Las rutas por defecto apuntan únicamente "hacia arriba"**: PC → router de LAN → leaf → spine.
2. **Hacia abajo se usan rutas específicas**: cada leaf conoce las redes de sus routers; cada spine conoce qué leaf alcanza cada red.
3. **Los spines no tienen ruta por defecto.** Como la topología no tiene salida a Internet, el tráfico hacia un destino desconocido debe descartarse en el nivel superior en lugar de rebotar entre spines.

No fue necesario agregar un enlace directo entre spine1 y spine2: las redes Net1 a Net4 cuelgan de leafs conectados a ambos spines, por lo que alcanzan Net5 y Net6 directamente. El único par que requiere cruzar de un spine al otro es **Net5 ↔ Net6**, y se resuelve con una ruta específica a través de leaf1.

Las redes de tránsito

Una primera versión de las tablas de los spines incluía únicamente las LANs. Al probar conectividad extremo a extremo entre `n1` y `n24` el ping fallaba con 100 % de pérdida, pero el mismo ping forzando la dirección de origen a la LAN (`ping -I 16.0.2.1`) funcionaba con TTL 61.

El diagnóstico es que **el camino de ida era correcto y el de vuelta no**: al enviar el datagrama, `n1` lo emite con dirección de origen `16.0.6.1` —la interfaz del enlace punto a punto, porque es la que usa para salir—. `n24` responde a esa dirección, pero al llegar la respuesta a spine1, este no tenía ruta hacia la red `16.0.6.0/30` y descartaba el datagrama.

Se agregaron entonces las rutas hacia las **redes de tránsito**. El direccionamiento permitió **sumarizarlas**: los dos enlaces de cada leaf quedaron contiguos y alineados, por lo que cada par de `/30` se agrupa en un único `/29`.

Leaf	Enlaces hacia abajo	Enlaces hacia arriba
leaf1	`16.0.6.0/30` + `16.0.6.4/30` → **`16.0.6.0/29`**	`16.0.6.32/30` + `16.0.6.36/30` → **`16.0.6.32/29`**
leaf2	`16.0.6.8/30` + `16.0.6.12/30` → **`16.0.6.8/29`**	`16.0.6.40/30` + `16.0.6.44/30` → **`16.0.6.40/29`**
leaf3	`16.0.6.16/30` + `16.0.6.20/30` → **`16.0.6.16/29`**	`16.0.6.48/30` + `16.0.6.52/30` → **`16.0.6.48/29`**
leaf4	`16.0.6.24/30` + `16.0.6.28/30` → **`16.0.6.24/29`**	`16.0.6.56/30` + `16.0.6.60/30` → **`16.0.6.56/29`**

De este modo cada spine incorpora **8 rutas** en lugar de 16, y la tabla refleja la jerarquía de la topología: cada `/29` identifica a un leaf.

Los leafs no necesitan estas rutas: alcanzan los enlaces de las otras ramas por su ruta por defecto hacia spine1.

Tablas de ruteo IPv4

Routers de LAN

n1

Destino	Next hop	Máscara	Interfaz
16.0.2.0/24 (su LAN)	–	255.255.255.0	eth1
16.0.6.0/30 (n1–leaf1)	–	255.255.255.252	eth0
default	**16.0.6.2** (leaf1)	0.0.0.0	eth0

n2

Destino	Next hop	Máscara	Interfaz
16.0.3.0/24 (su LAN)	–	255.255.255.0	eth1
16.0.6.4/30 (n2–leaf1)	–	255.255.255.252	eth0
default	**16.0.6.6** (leaf1)	0.0.0.0	eth0

n17

Destino	Next hop	Máscara	Interfaz
16.0.4.0/25 (su LAN)	–	255.255.255.128	eth0
16.0.6.8/30 (n17–leaf2)	–	255.255.255.252	eth1
default	**16.0.6.10** (leaf2)	0.0.0.0	eth1

n18

Destino	Next hop	Máscara	Interfaz
16.0.5.64/27 (su LAN)	–	255.255.255.224	eth0
16.0.6.12/30 (n18–leaf2)	–	255.255.255.252	eth1
default	**16.0.6.14** (leaf2)	0.0.0.0	eth1

n21

Destino	Next hop	Máscara	Interfaz
16.0.4.128/25 (su LAN)	–	255.255.255.128	eth0
16.0.6.16/30 (n21–leaf3)	–	255.255.255.252	eth1
default	**16.0.6.18** (leaf3)	0.0.0.0	eth1

n22

Destino	Next hop	Máscara	Interfaz
16.0.5.0/26 (su LAN)	–	255.255.255.192	eth0
16.0.6.20/30 (n22–leaf3)	–	255.255.255.252	eth1
default	**16.0.6.22** (leaf3)	0.0.0.0	eth1

n23

Destino	Next hop	Máscara	Interfaz
16.0.5.96/27 (su LAN)	–	255.255.255.224	eth0
16.0.6.24/30 (n23–leaf4)	–	255.255.255.252	eth1
default	**16.0.6.26** (leaf4)	0.0.0.0	eth1

n24

Destino	Next hop	Máscara	Interfaz
16.0.5.128/27 (su LAN)	–	255.255.255.224	eth0
16.0.6.28/30 (n24–leaf4)	–	255.255.255.252	eth1
default	**16.0.6.30** (leaf4)	0.0.0.0	eth1

Leafs

leaf1

Destino	Next hop	Máscara	Interfaz
16.0.6.0/30 (n1–leaf1)	–	255.255.255.252	eth0
16.0.6.4/30 (n2–leaf1)	–	255.255.255.252	eth1
16.0.6.32/30 (leaf1–spine1)	–	255.255.255.252	eth2
16.0.6.36/30 (leaf1–spine2)	–	255.255.255.252	eth3
16.0.2.0/24 (Net1-A, vía n1)	16.0.6.1	255.255.255.0	eth0
16.0.3.0/24 (Net1-B, vía n2)	16.0.6.5	255.255.255.0	eth1
16.0.5.160/28 (Net6, vía spine2)	16.0.6.38	255.255.255.240	eth3
default	**16.0.6.34** (spine1)	0.0.0.0	eth2

leaf2

Destino	Next hop	Máscara	Interfaz
16.0.6.8/30 (n17–leaf2)	–	255.255.255.252	eth0
16.0.6.12/30 (n18–leaf2)	–	255.255.255.252	eth1
16.0.6.44/30 (leaf2–spine2)	–	255.255.255.252	eth2
16.0.6.40/30 (leaf2–spine1)	–	255.255.255.252	eth3
16.0.4.0/25 (Net2-A, vía n17)	16.0.6.9	255.255.255.128	eth0
16.0.5.64/27 (Net2-B, vía n18)	16.0.6.13	255.255.255.224	eth1
default	**16.0.6.42** (spine1)	0.0.0.0	eth3

leaf3

Destino	Next hop	Máscara	Interfaz
16.0.6.16/30 (n21–leaf3)	–	255.255.255.252	eth0
16.0.6.20/30 (n22–leaf3)	–	255.255.255.252	eth1
16.0.6.52/30 (leaf3–spine2)	–	255.255.255.252	eth2
16.0.6.48/30 (leaf3–spine1)	–	255.255.255.252	eth3
16.0.4.128/25 (Net3-A, vía n21)	16.0.6.17	255.255.255.128	eth0
16.0.5.0/26 (Net3-B, vía n22)	16.0.6.21	255.255.255.192	eth1
default	**16.0.6.50** (spine1)	0.0.0.0	eth3

leaf4

Destino	Next hop	Máscara	Interfaz
16.0.6.24/30 (n23–leaf4)	–	255.255.255.252	eth0
16.0.6.28/30 (n24–leaf4)	–	255.255.255.252	eth1
16.0.6.60/30 (leaf4–spine2)	–	255.255.255.252	eth2
16.0.6.56/30 (leaf4–spine1)	–	255.255.255.252	eth3
16.0.5.96/27 (Net4-A, vía n23)	16.0.6.25	255.255.255.224	eth0
16.0.5.128/27 (Net4-B, vía n24)	16.0.6.29	255.255.255.224	eth1
default	**16.0.6.58** (spine1)	0.0.0.0	eth3

Spines

spine1 — sin ruta por defecto

Destino	Next hop	Máscara	Interfaz
16.0.6.32/30 (leaf1)	–	255.255.255.252	eth0
16.0.0.0/23 (LAN Net5)	–	255.255.254.0	eth1
16.0.6.40/30 (leaf2)	–	255.255.255.252	eth2
16.0.6.48/30 (leaf3)	–	255.255.255.252	eth3
16.0.6.56/30 (leaf4)	–	255.255.255.252	eth4
16.0.2.0/24 (Net1-A)	16.0.6.33 (leaf1)	255.255.255.0	eth0
16.0.3.0/24 (Net1-B)	16.0.6.33 (leaf1)	255.255.255.0	eth0
16.0.4.0/25 (Net2-A)	16.0.6.41 (leaf2)	255.255.255.128	eth2
16.0.5.64/27 (Net2-B)	16.0.6.41 (leaf2)	255.255.255.224	eth2
16.0.4.128/25 (Net3-A)	16.0.6.49 (leaf3)	255.255.255.128	eth3
16.0.5.0/26 (Net3-B)	16.0.6.49 (leaf3)	255.255.255.192	eth3
16.0.5.96/27 (Net4-A)	16.0.6.57 (leaf4)	255.255.255.224	eth4
16.0.5.128/27 (Net4-B)	16.0.6.57 (leaf4)	255.255.255.224	eth4
16.0.5.160/28 (Net6)	16.0.6.33 (leaf1 → spine2)	255.255.255.240	eth0
16.0.6.0/29 (enlaces de leaf1)	16.0.6.33 (leaf1)	255.255.255.248	eth0
16.0.6.32/29 (enlaces de leaf1)	16.0.6.33 (leaf1)	255.255.255.248	eth0
16.0.6.8/29 (enlaces de leaf2)	16.0.6.41 (leaf2)	255.255.255.248	eth2
16.0.6.40/29 (enlaces de leaf2)	16.0.6.41 (leaf2)	255.255.255.248	eth2
16.0.6.16/29 (enlaces de leaf3)	16.0.6.49 (leaf3)	255.255.255.248	eth3
16.0.6.48/29 (enlaces de leaf3)	16.0.6.49 (leaf3)	255.255.255.248	eth3
16.0.6.24/29 (enlaces de leaf4)	16.0.6.57 (leaf4)	255.255.255.248	eth4
16.0.6.56/29 (enlaces de leaf4)	16.0.6.57 (leaf4)	255.255.255.248	eth4

spine2 — sin ruta por defecto

Destino	Next hop	Máscara	Interfaz
16.0.6.60/30 (leaf4)	–	255.255.255.252	eth0
16.0.6.52/30 (leaf3)	–	255.255.255.252	eth1
16.0.5.160/28 (LAN Net6)	–	255.255.255.240	eth2
16.0.6.44/30 (leaf2)	–	255.255.255.252	eth3
16.0.6.36/30 (leaf1)	–	255.255.255.252	eth4
16.0.2.0/24 (Net1-A)	16.0.6.37 (leaf1)	255.255.255.0	eth4
16.0.3.0/24 (Net1-B)	16.0.6.37 (leaf1)	255.255.255.0	eth4
16.0.4.0/25 (Net2-A)	16.0.6.45 (leaf2)	255.255.255.128	eth3
16.0.5.64/27 (Net2-B)	16.0.6.45 (leaf2)	255.255.255.224	eth3
16.0.4.128/25 (Net3-A)	16.0.6.53 (leaf3)	255.255.255.128	eth1
16.0.5.0/26 (Net3-B)	16.0.6.53 (leaf3)	255.255.255.192	eth1
16.0.5.96/27 (Net4-A)	16.0.6.61 (leaf4)	255.255.255.224	eth0
16.0.5.128/27 (Net4-B)	16.0.6.61 (leaf4)	255.255.255.224	eth0
16.0.0.0/23 (Net5)	16.0.6.37 (leaf1 → spine1)	255.255.254.0	eth4
16.0.6.0/29 (enlaces de leaf1)	16.0.6.37 (leaf1)	255.255.255.248	eth4
16.0.6.32/29 (enlaces de leaf1)	16.0.6.37 (leaf1)	255.255.255.248	eth4
16.0.6.8/29 (enlaces de leaf2)	16.0.6.45 (leaf2)	255.255.255.248	eth3
16.0.6.40/29 (enlaces de leaf2)	16.0.6.45 (leaf2)	255.255.255.248	eth3
16.0.6.16/29 (enlaces de leaf3)	16.0.6.53 (leaf3)	255.255.255.248	eth1
16.0.6.48/29 (enlaces de leaf3)	16.0.6.53 (leaf3)	255.255.255.248	eth1
16.0.6.24/29 (enlaces de leaf4)	16.0.6.61 (leaf4)	255.255.255.248	eth0
16.0.6.56/29 (enlaces de leaf4)	16.0.6.61 (leaf4)	255.255.255.248	eth0

Tabla de ruteo IPv6

Al resolverse la conectividad IPv6 mediante el túnel, el ruteo IPv6 se reduce a una ruta estática en cada spine.

spine1

Destino	Next hop	Interfaz
`1600:0:0:5::/64` (LAN Net5)	–	eth4
`1600:0:0:ffff::/64` (túnel)	–	tun6
`1600:0:0:6::/64` (LAN Net6)	– *(on-link por el túnel)*	tun6

spine2

Destino	Next hop	Interfaz
`1600:0:0:6::/64` (LAN Net6)	–	eth4
`1600:0:0:ffff::/64` (túnel)	–	tun6
`1600:0:0:5::/64` (LAN Net5)	– *(on-link por el túnel)*	tun6

n34 y n35 llevan únicamente la ruta por defecto hacia el spine de su LAN:

```
# n34
ip -6 route add default via 1600:0:0:5::1
# n35
ip -6 route add default via 1600:0:0:6::1
```

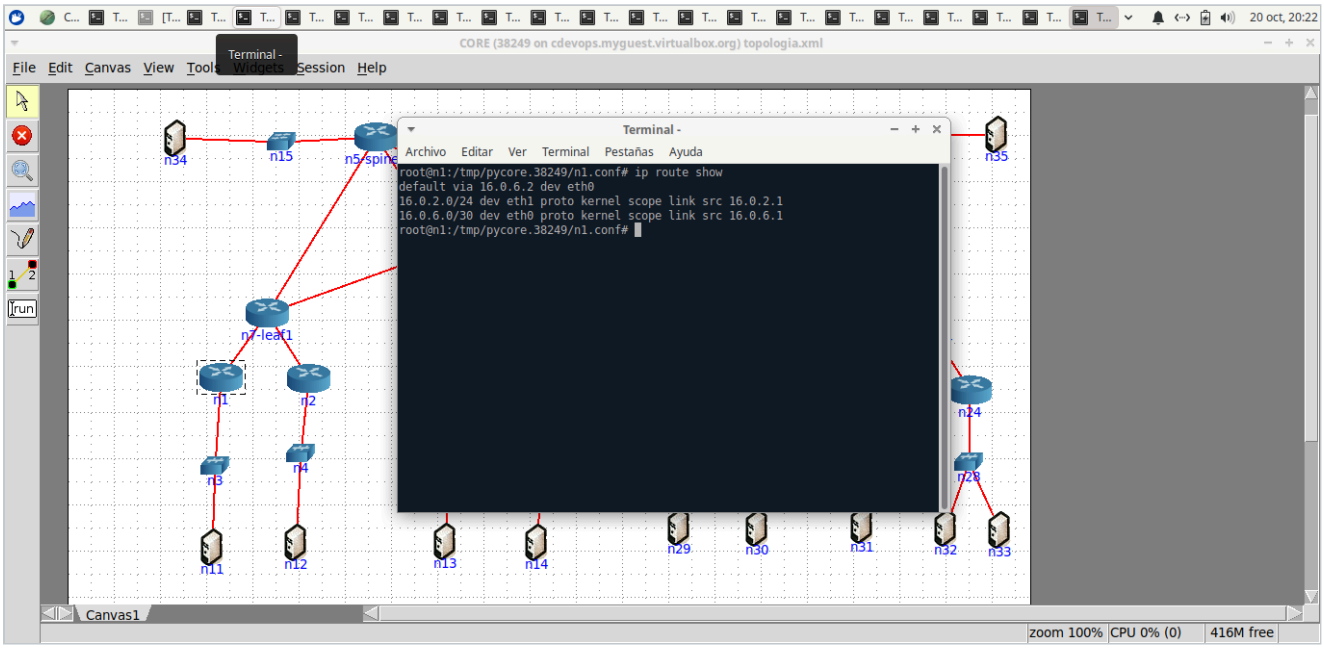


*Figura B.1 — `ip route show` en n1***

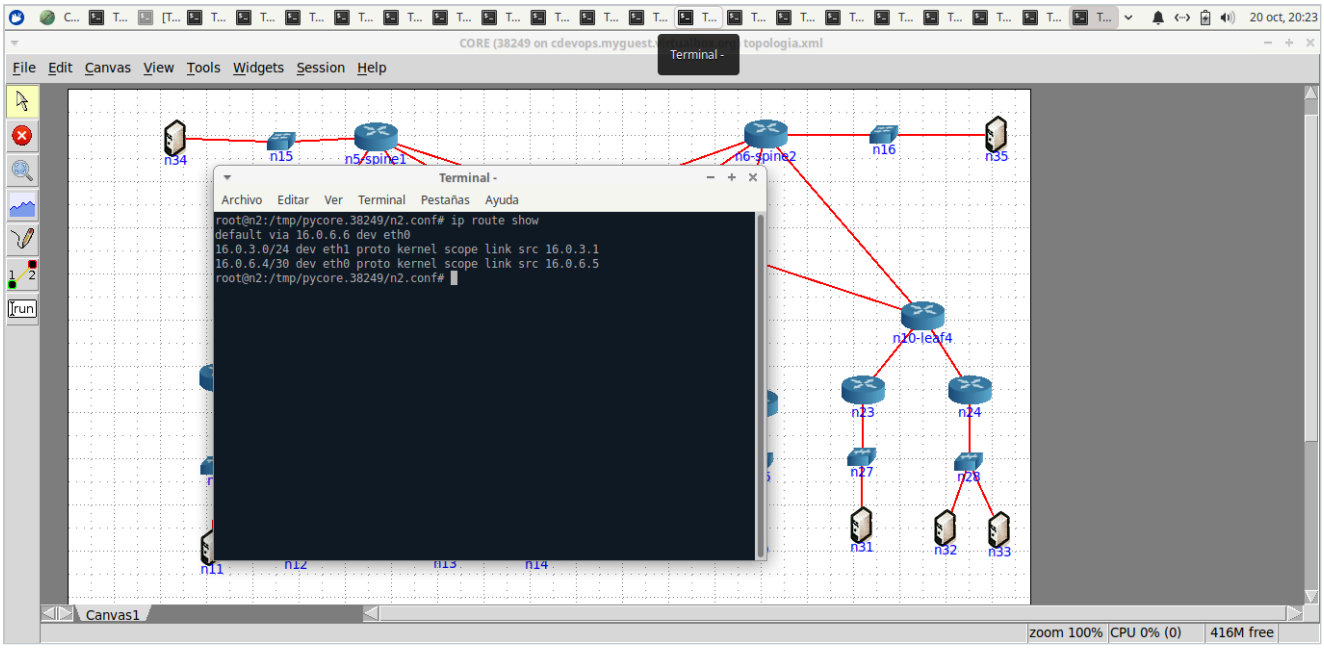


*Figura B.2 — `ip route show` en n2***

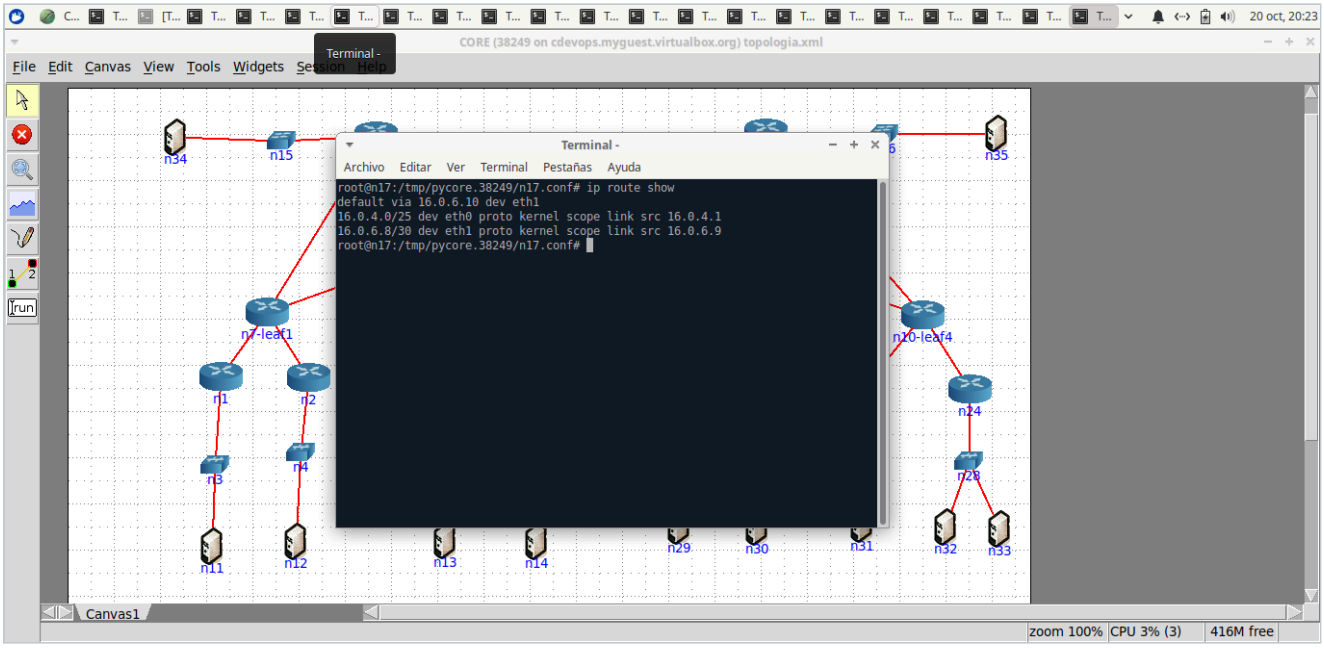


*Figura B.3 — `ip route show` en n17***

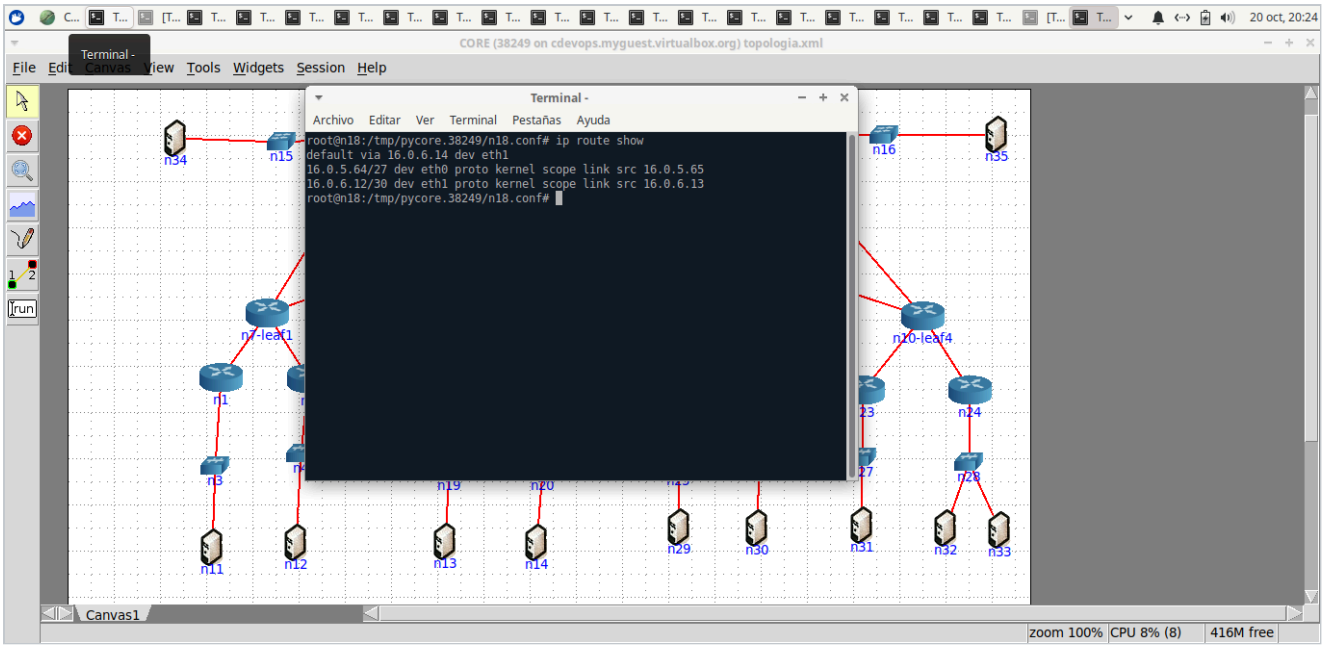


*Figura B.4 — `ip route show` en n18***

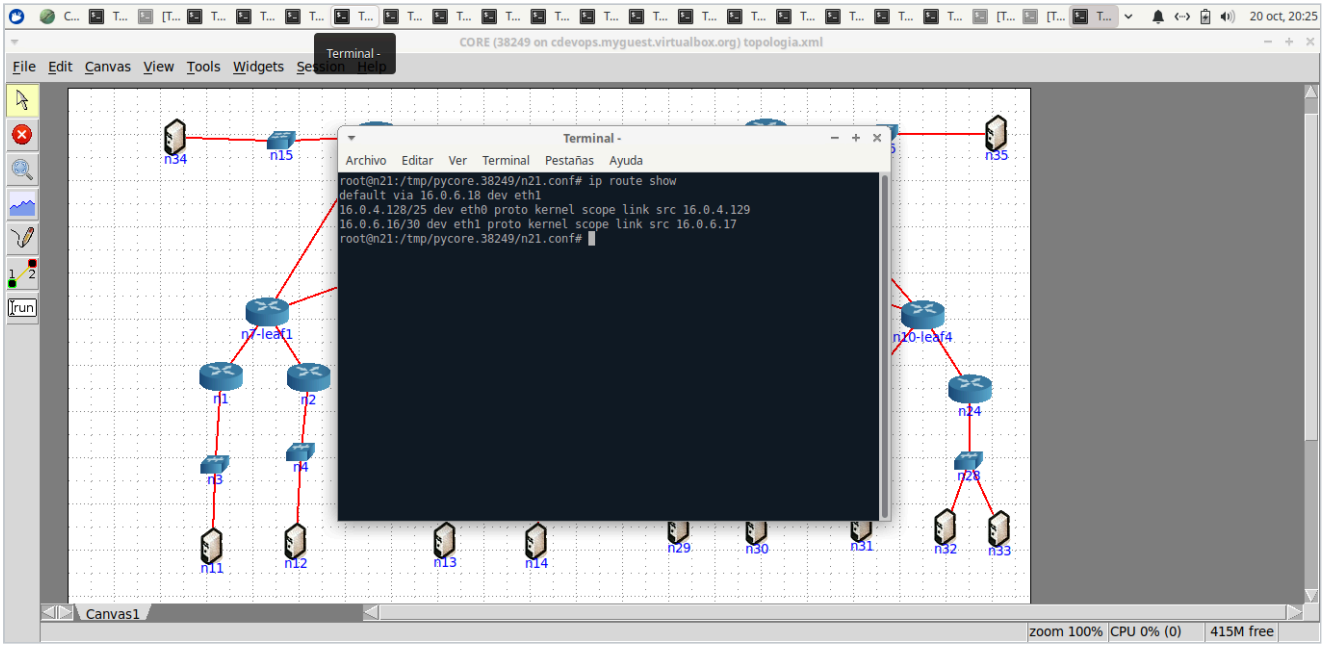


*Figura B.5 — `ip route show` en n21***

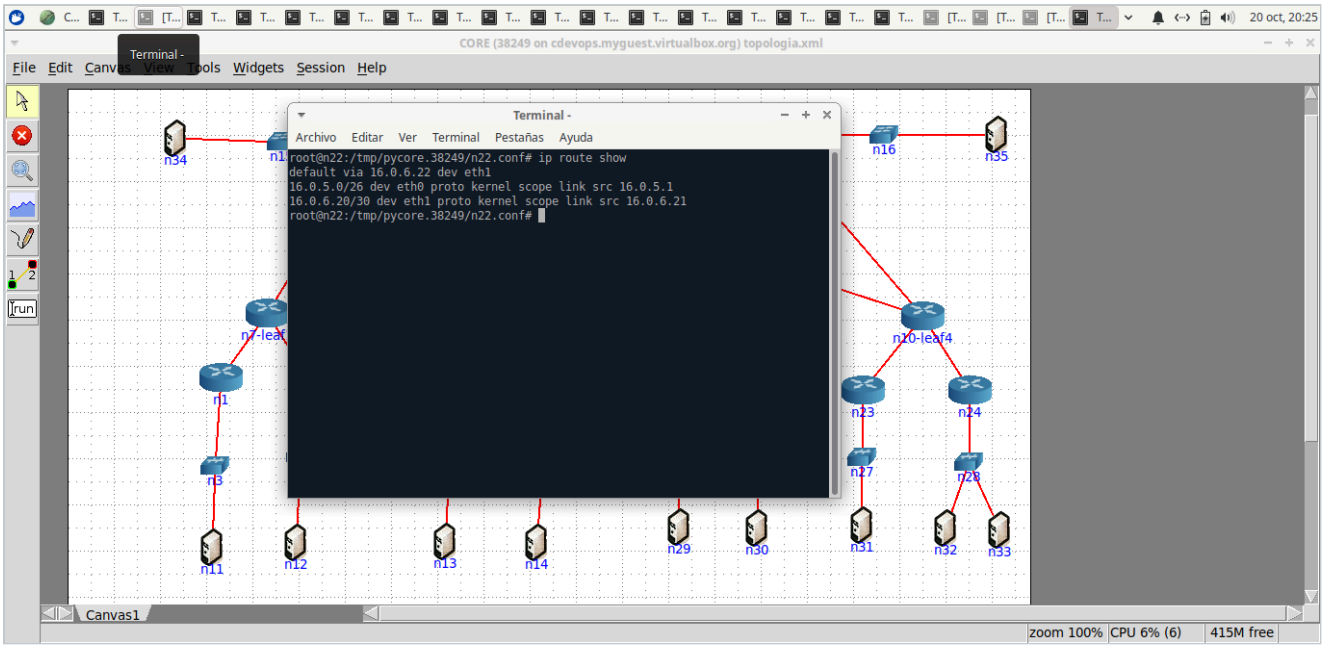


*Figura B.6 — `ip route show` en n22***

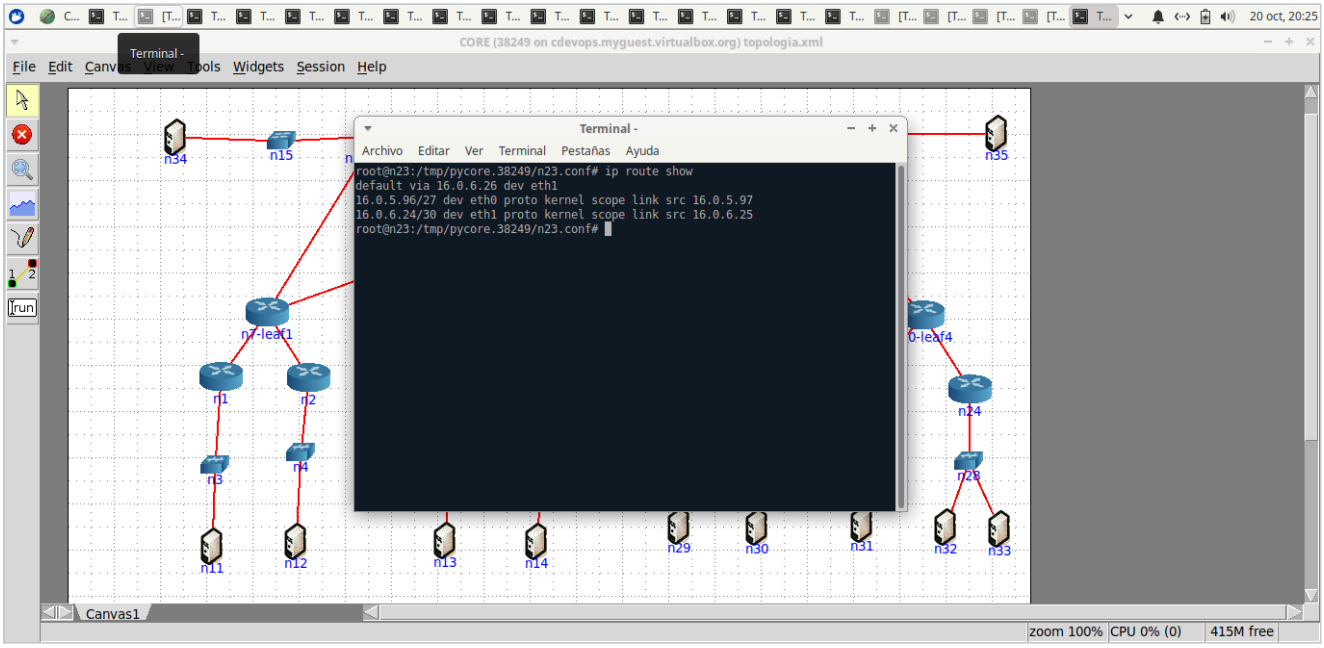


*Figura B.7 — `ip route show` en n23***

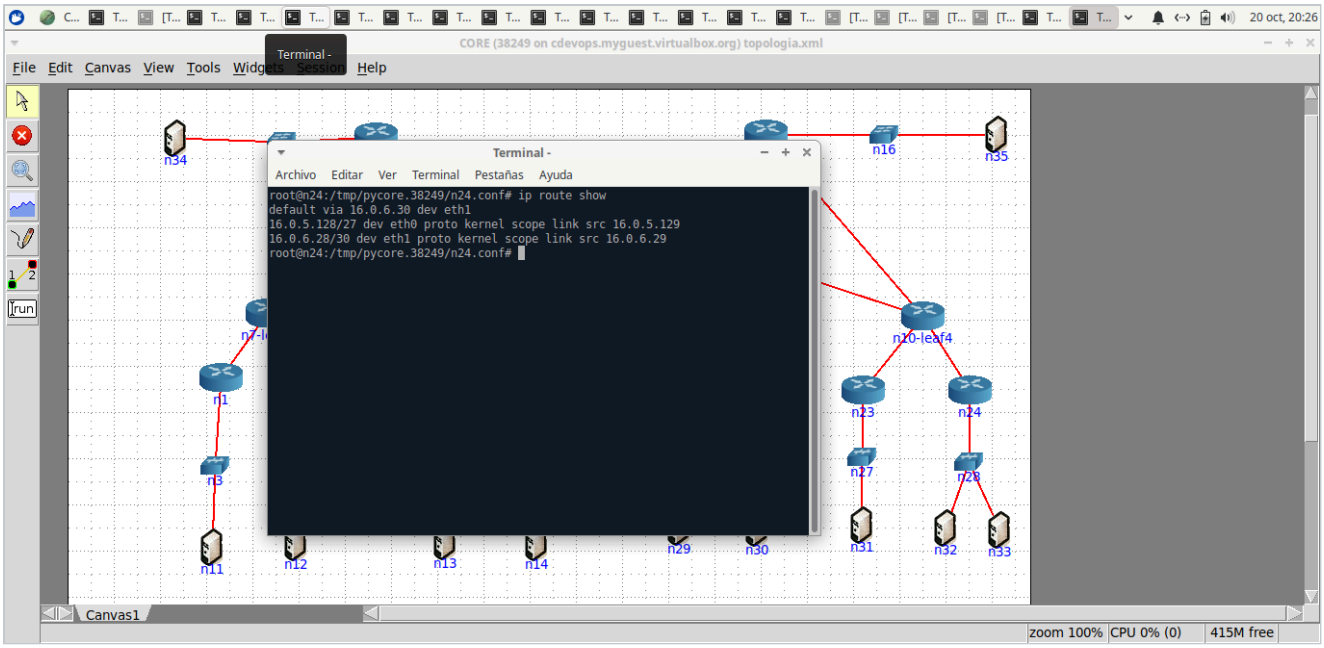


*Figura B.8 — `ip route show` en n24***

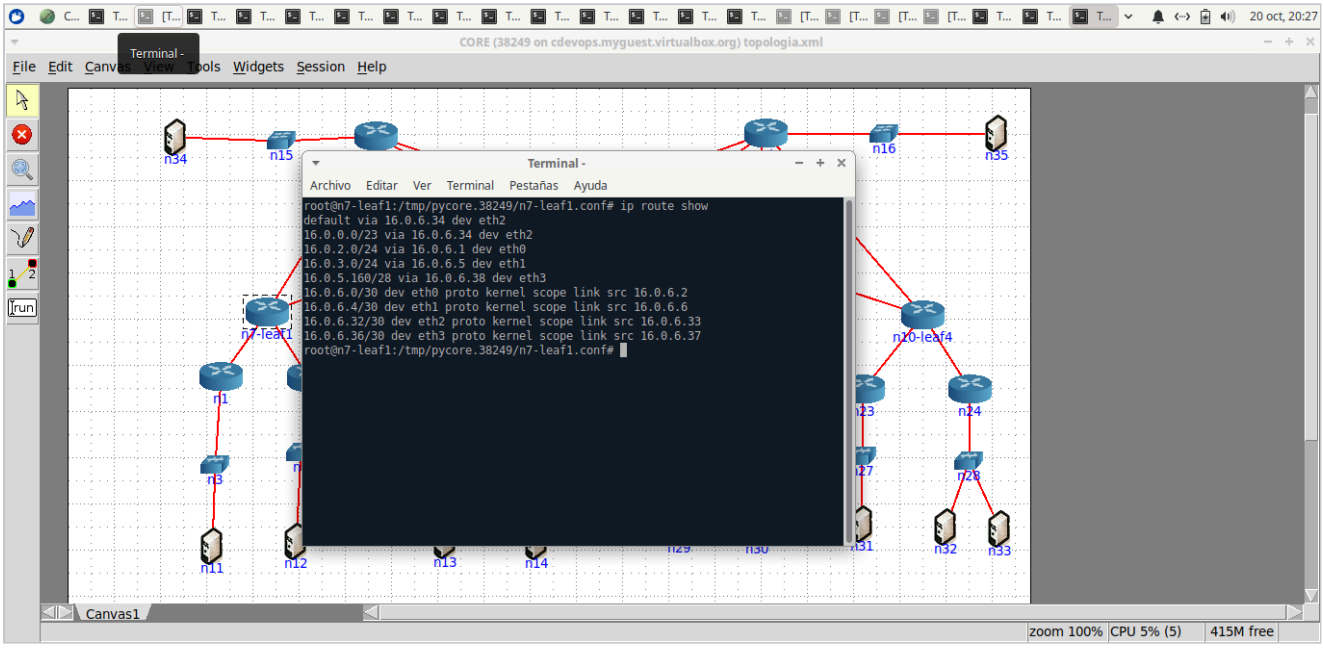


*Figura B.9 — `ip route show` en leaf1***

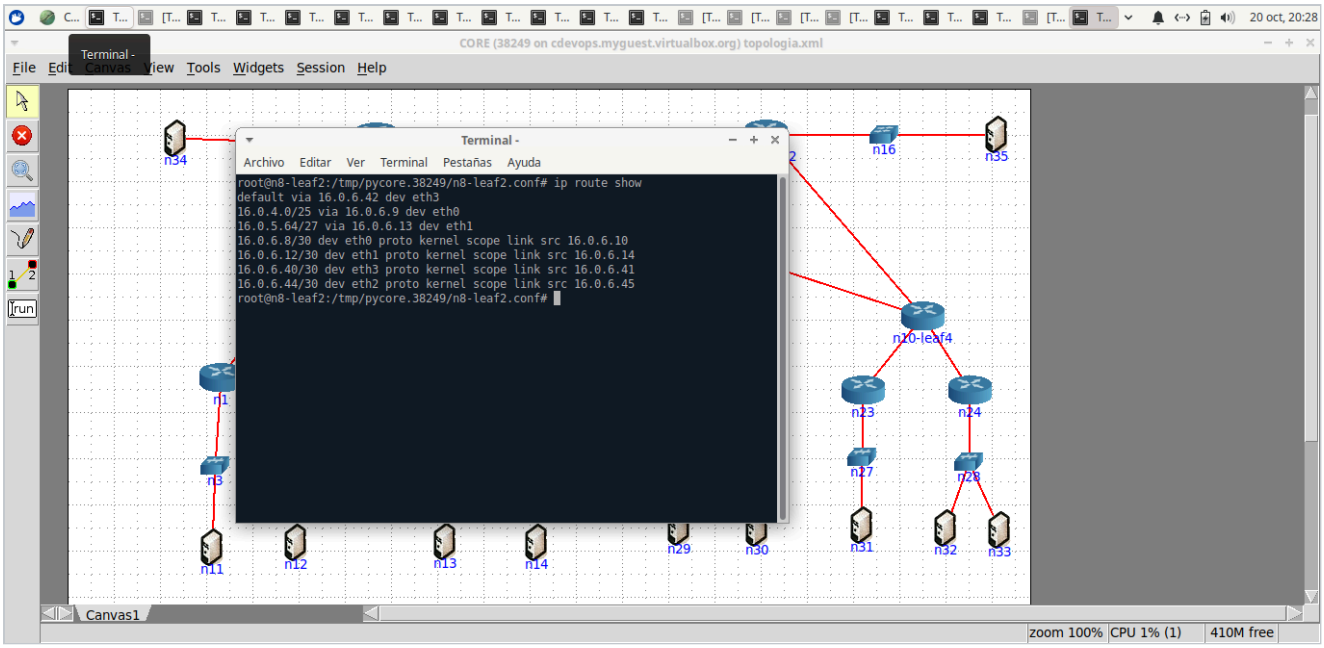


*Figura B.10 — `ip route show` en leaf2***

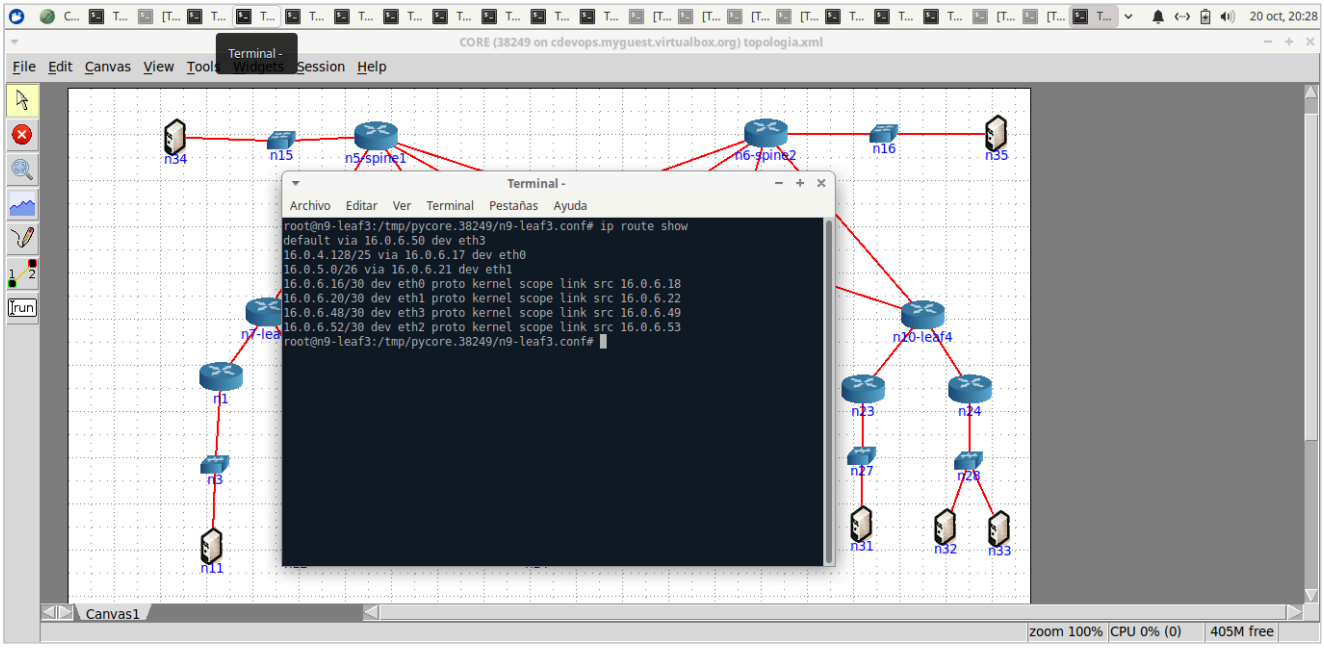


*Figura B.11 — `ip route show` en leaf3***

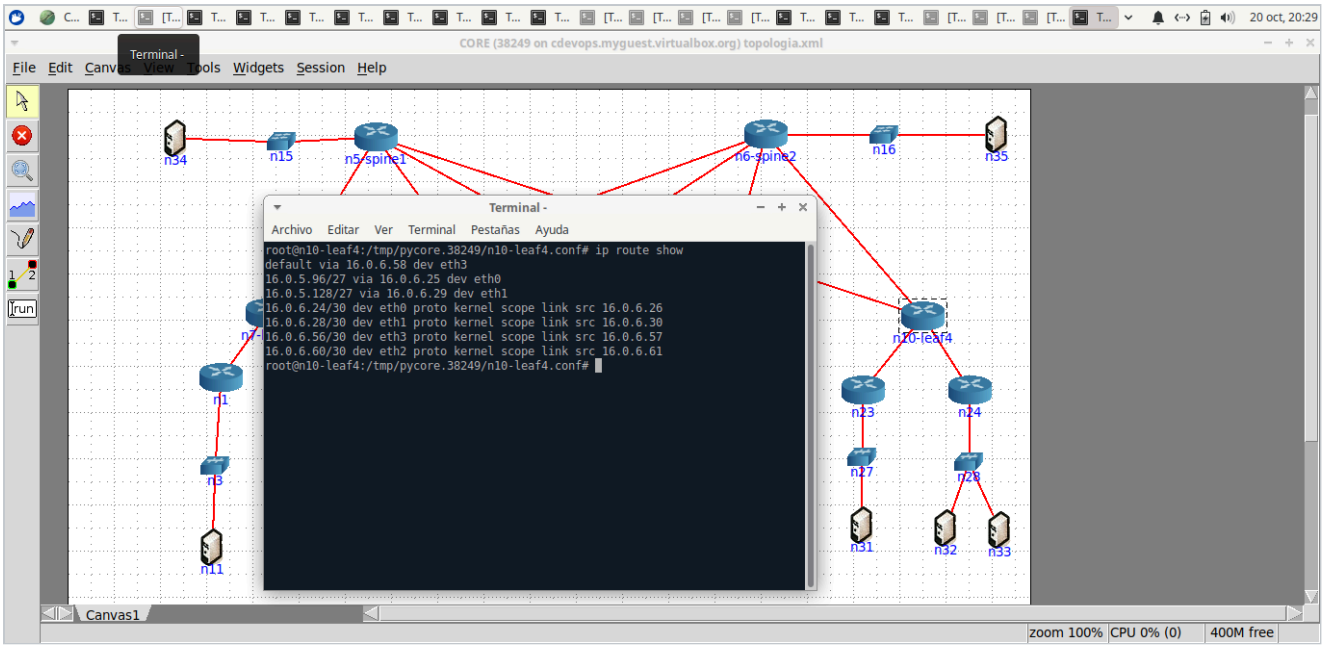


*Figura B.12 — `ip route show` en leaf4***

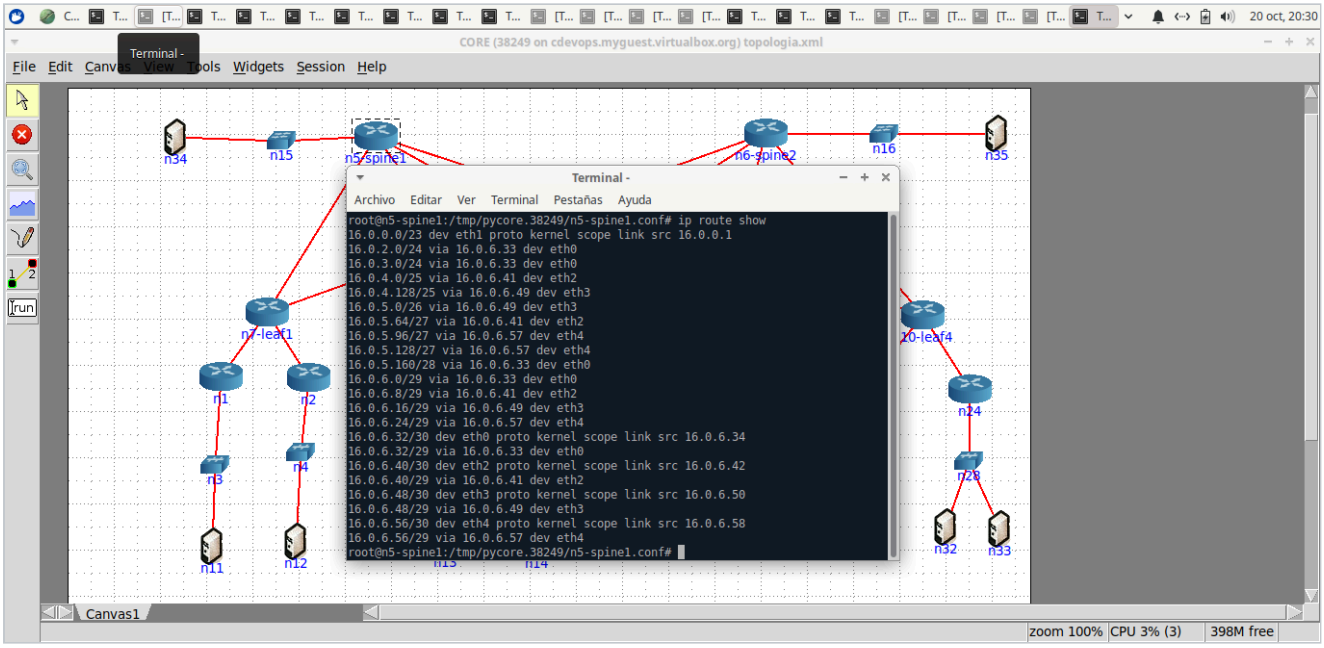


*Figura B.13 — `ip route show` en spine1***

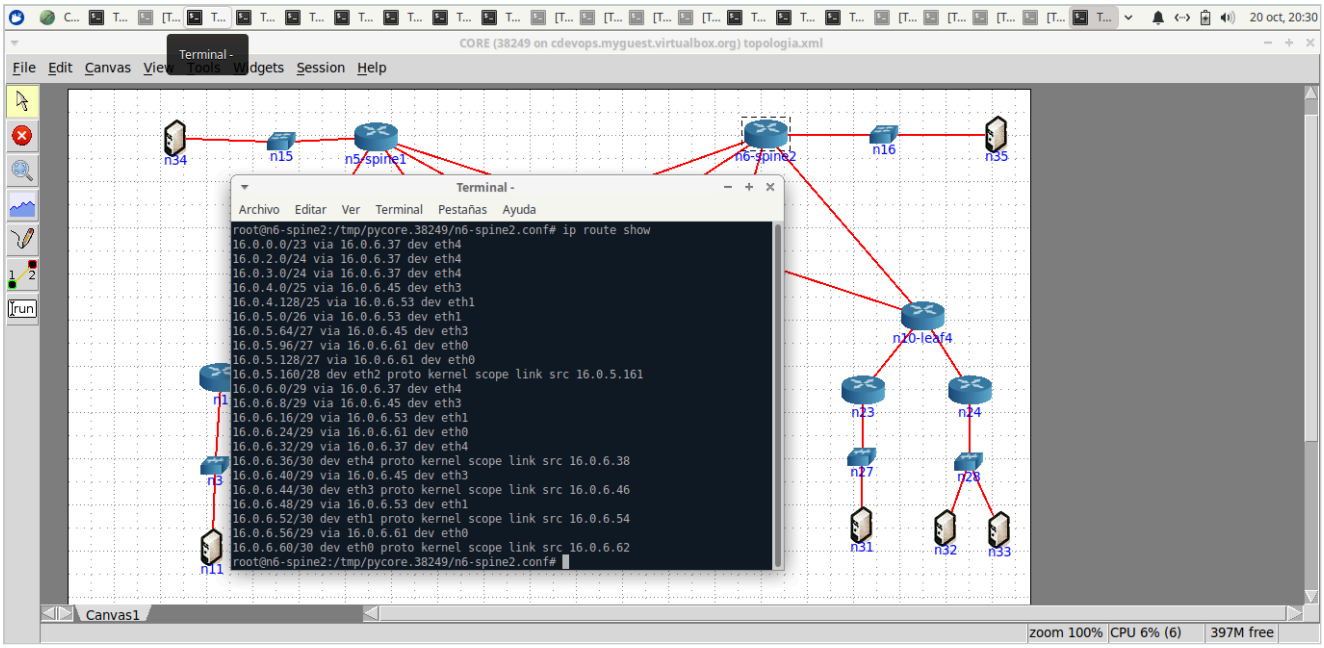


*Figura B.14 — `ip route show` en spine2***

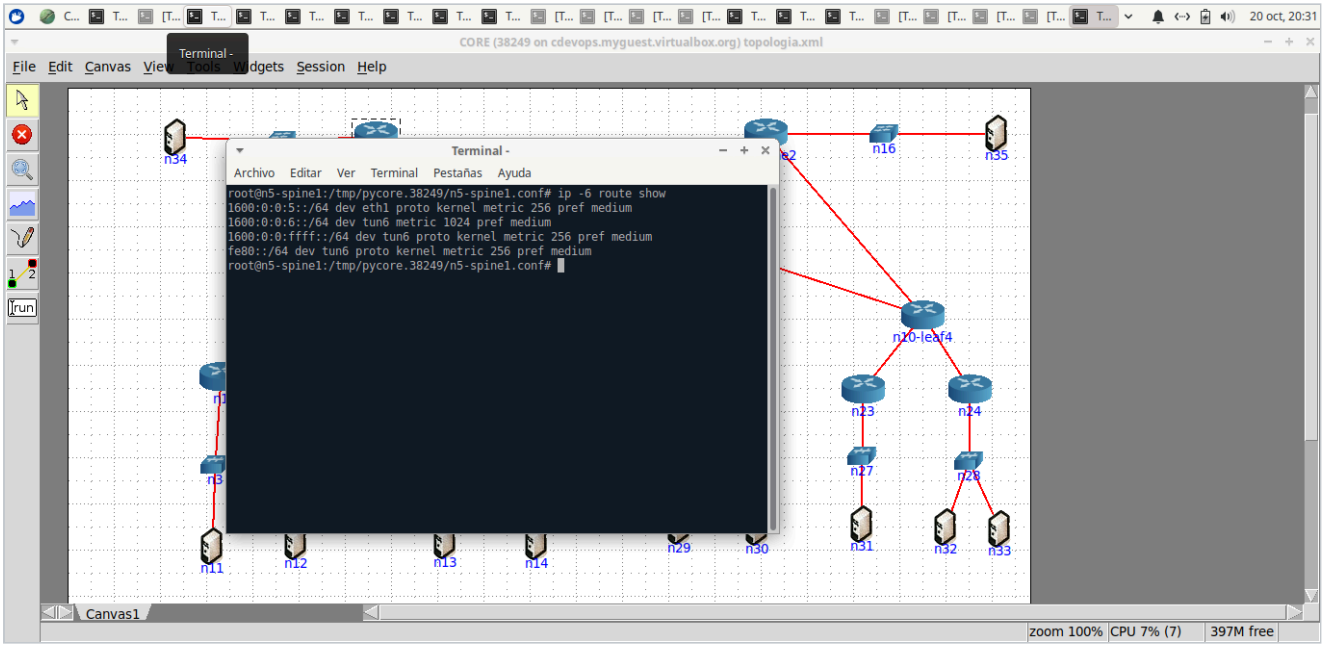


Figura B.15 — `ip -6 route show` en spine1

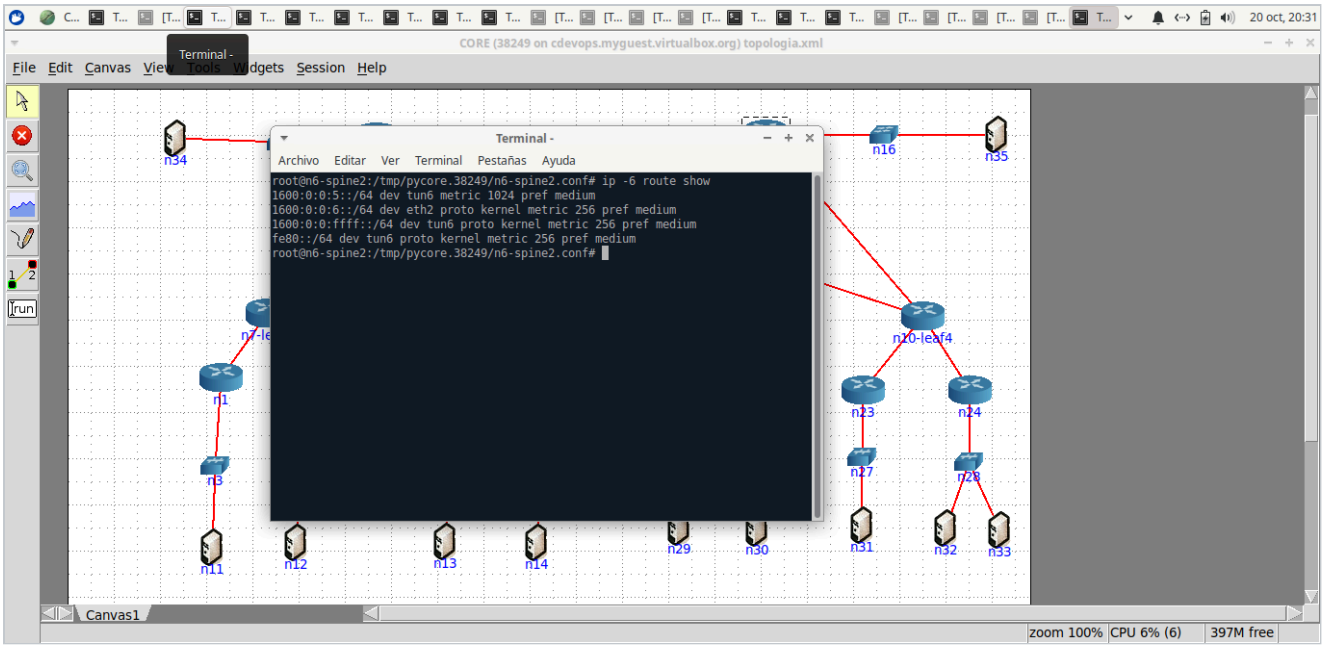


Figura B.16 — `ip -6 route show` en spine2

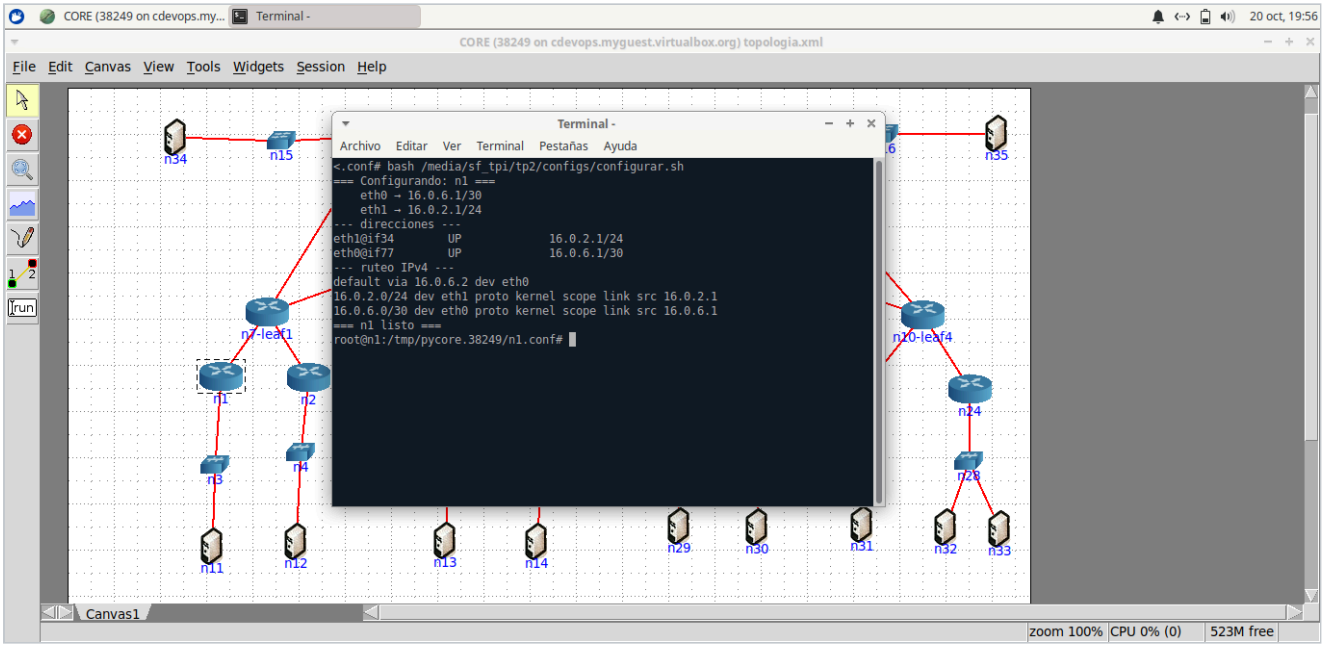


Figura B.17 — ejecución de `configurar.sh` en n1: el script imprime direcciones y tabla de ruteo al finalizar

---

Inciso D — Test con ping y traceroute

Con el direccionamiento y las rutas configuradas, se verificó la conectividad entre nodos adyacentes y entre extremos de la topología con `ping` y `traceroute`.

```
ping -c 4 16.0.2.1          # n11 → su gateway
ping -c 4 16.0.6.2          # n11 → leaf1
ping -c 4 16.0.5.130        # n11 → n33, atraviesa toda la topología
traceroute 16.0.5.130
```

Verificación de ausencia de bucles. Se hace ping a una dirección que no existe en la topología:

```
ping 1.2.3.4
```

- Respuesta **ICMP Destination Unreachable, Net Unreachable (tipo 3, código 0)** → correcto: algún router descartó el datagrama por falta de ruta.
- Respuesta **ICMP Time Exceeded (tipo 11, código 0)** → indicaría que el datagrama circuló hasta agotar el TTL, es decir, **hay un bucle de ruteo**.

Figuras:

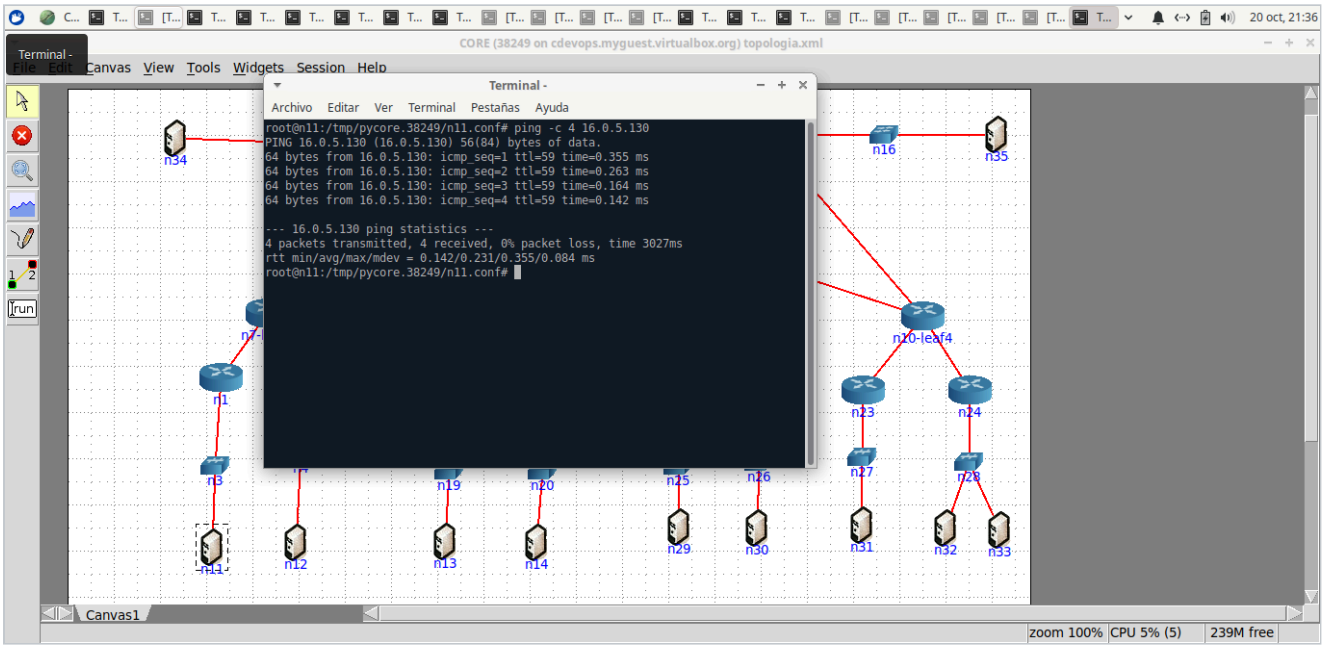


Figura D.1 — ping de n11 a n33: 0% de pérdida y TTL 59 (64 − 5 routers)

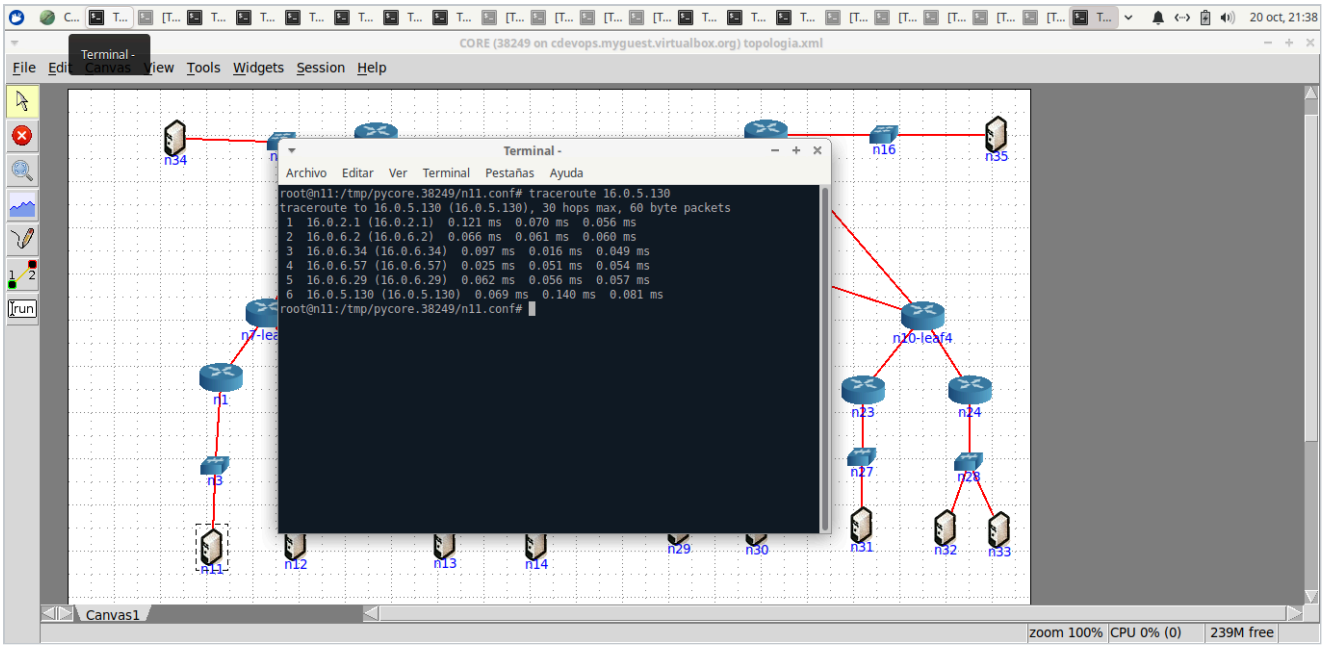


Figura D.2 — traceroute de n11 a n33: n1 → leaf1 → spine1 → leaf4 → n24 → n33

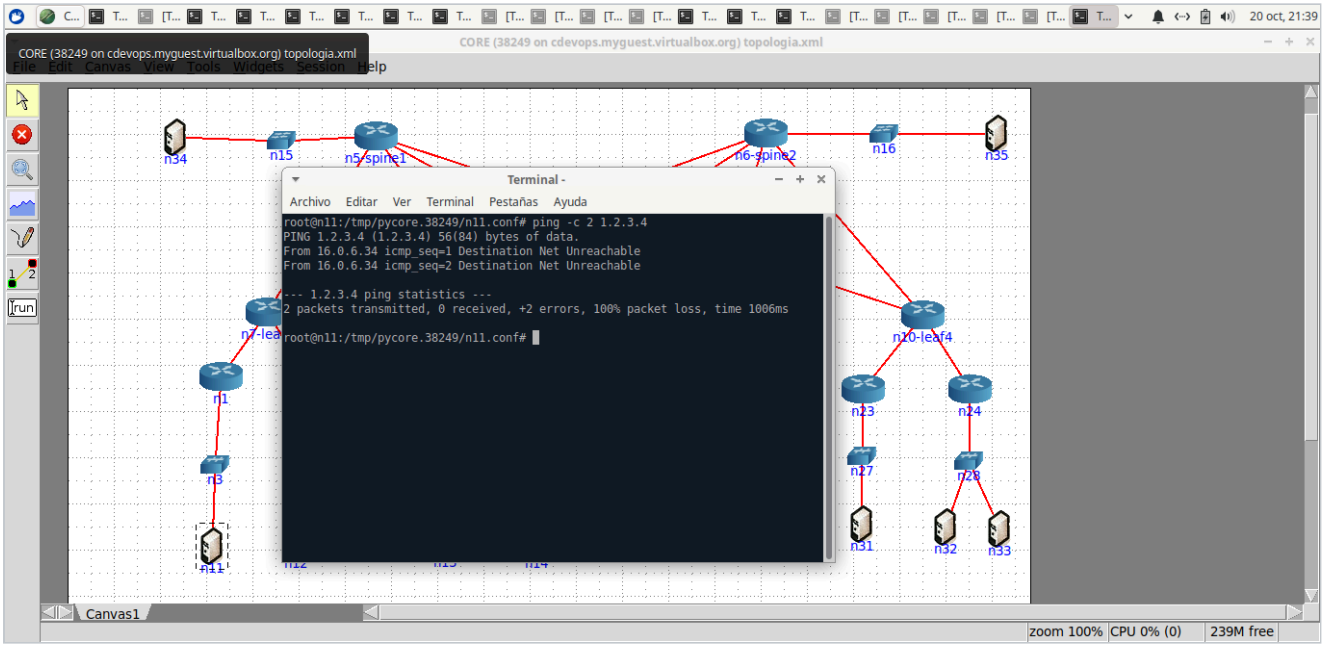


Figura D.3 — ping a `1.2.3.4`: Net Unreachable desde `16.0.6.34` (spine1) y no Time Exceeded. Confirma que no hay bucles y que el descarte ocurre en el spine, que por diseño no tiene ruta por defecto

---

Inciso E — Captura del tráfico n11 → n33: ARP e ICMP

Se realiza un ping desde **n11** (`16.0.2.10`, Net1-A) hacia **n33** (`16.0.5.130`, Net4-B) y se captura el tráfico en el segmento de n11.

```
tcpdump -i eth0 -w e-n11-n33.pcap 'arp or icmp'
```

Como n11 y n33 **no pertenecen a la misma red**, el Echo Request debe enviarse al default gateway. Antes, n11 necesita la MAC de ese gateway, y para eso emite un **ARP Request**.

ARP Request — trama Ethernet con EtherType `0x0806`:

Campo	Valor
MAC destino	`FF:FF:FF:FF:FF:FF` (broadcast)
MAC origen	MAC de n11
IP origen	`16.0.2.10` (n11)
IP destino	**`16.0.2.1` (n1)** ← el gateway, **no** n33

ARP Reply — respuesta de n1, en unicast:

Campo	Valor
MAC destino	MAC de n11
MAC origen	MAC de n1
IP origen	`16.0.2.1`
IP destino	`16.0.2.10`

Recién entonces n11 emite el **ICMP Echo Request (tipo 8, código 0)**, encapsulado en una trama dirigida a la MAC de n1 pero con **IP destino** **`16.0.5.130`**.

Observación central: la dirección IP de destino se mantiene constante de extremo a extremo, mientras que **las direcciones MAC se reescriben en cada salto**. Cada tramo del camino resuelve su propio ARP de forma independiente. n11 nunca conoce la MAC de n33.

El camino completo del datagrama es:

```
n11 → n3(hub) → n1 → leaf1 → spine(1 o 2) → leaf4 → n24 → n28(hub) → n33
```

Como el segmento de n11 cuelga de un **hub**, la captura puede tomarse desde cualquier PC del mismo segmento: al ser un único dominio de colisión, todas las estaciones reciben todas las tramas.

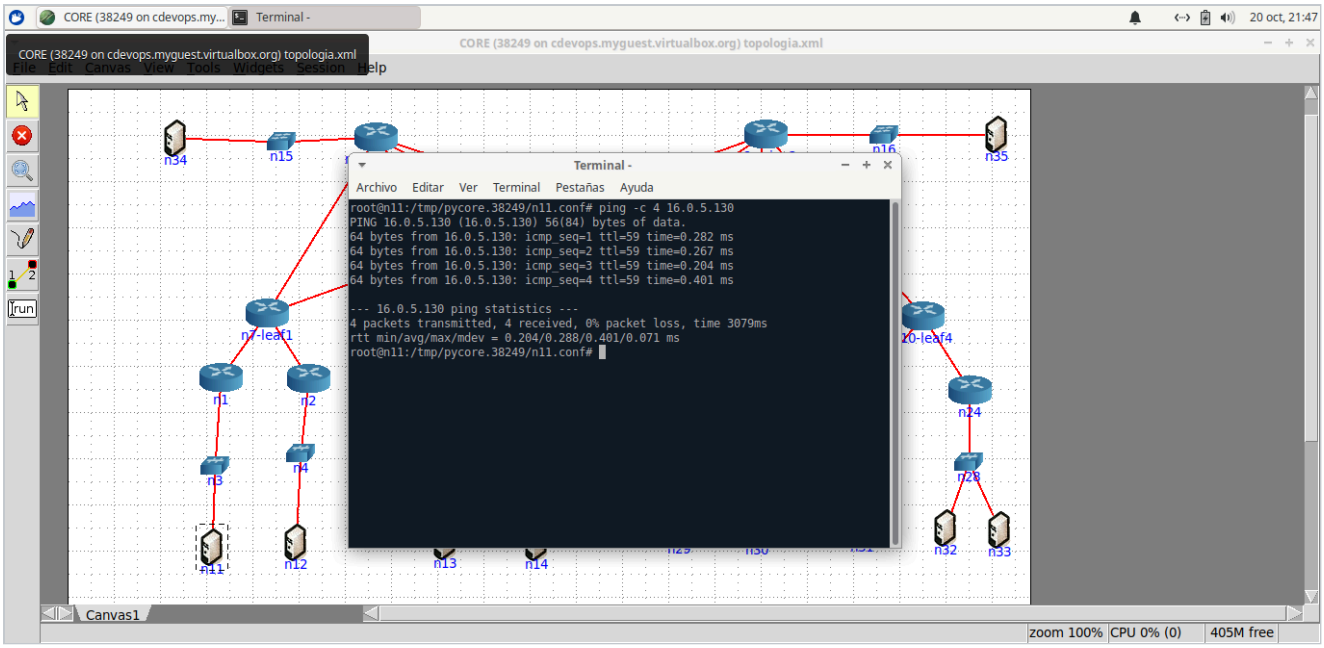


Figura E.1 — ping de n11 a n33 (TTL 59 = 64 − 5 routers)

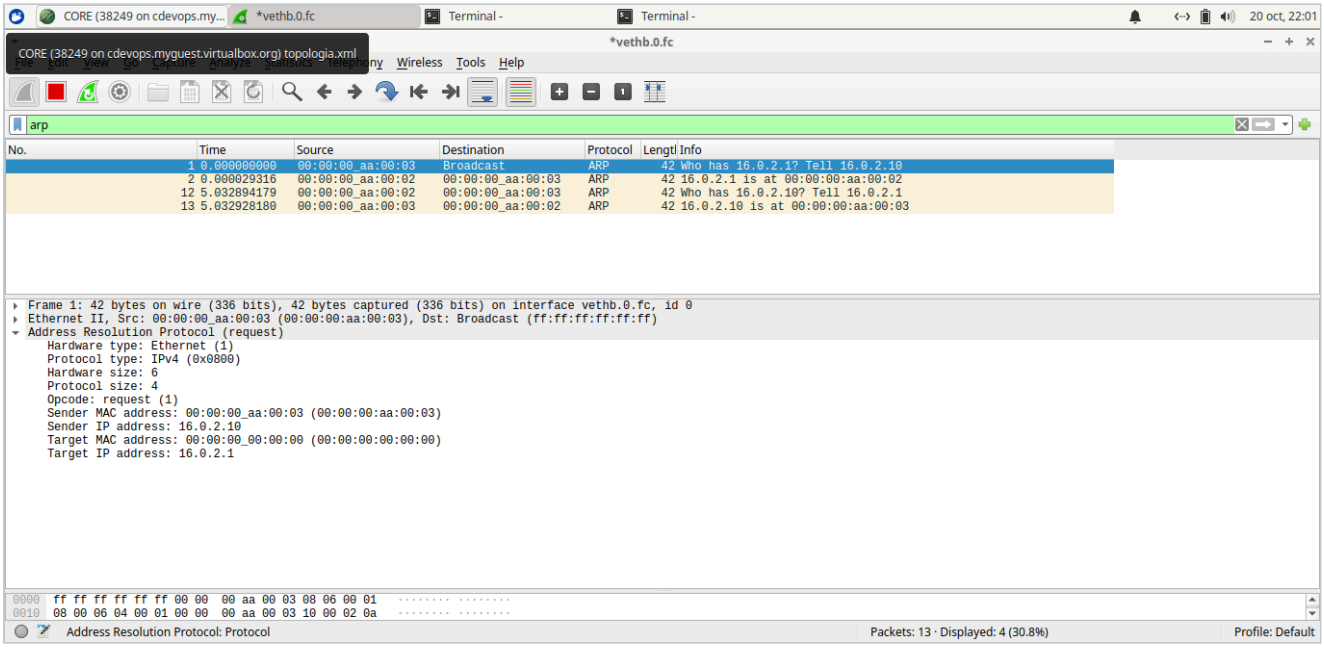
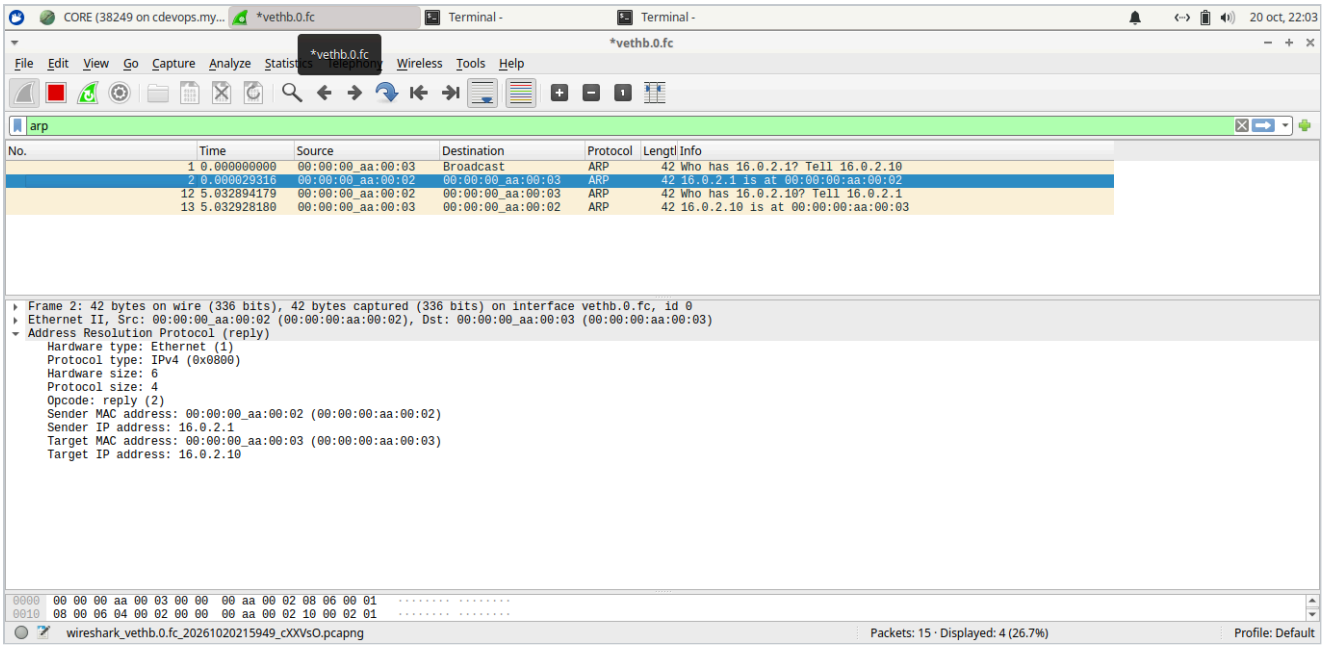
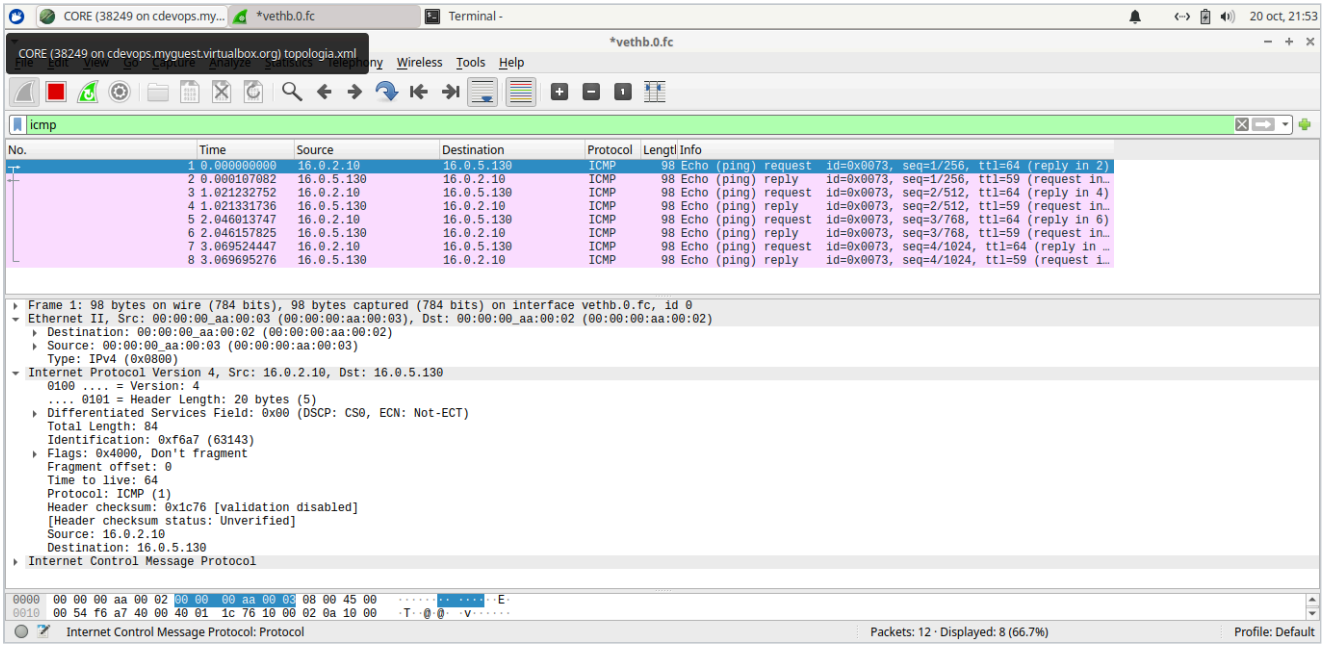


Figura E.2 — ARP Request: destino Ethernet `ff:ff:ff:ff:ff:ff` (broadcast) y `Target MAC` en cero. Es el paquete N° 1 de la captura: el ping no puede comenzar sin resolver antes la MAC



> *Figura E.3 — ARP Reply: unicast de n1 hacia n11, 29 µs después*



> *Figura E.4 — ICMP Echo Request: IP destino `16.0.5.130` (n33) pero MAC destino `00:00:00:aa:00:02` (n1, el gateway). Capa 2 local, capa 3 extremo a extremo*

Inciso F — Traceroute capturado

`traceroute` se apoya en el campo **TTL** del datagrama IPv4 y en los mensajes ICMP:

1. El origen envía una sonda con **TTL = 1**.
2. El primer router lo decrementa a 0, descarta el datagrama y devuelve al origen un **ICMP Time Exceeded — In Transit (tipo 11, código 0)**, revelando su dirección.
3. El origen repite con TTL = 2, 3, … y así descubre cada salto del camino.

4. Al llegar al destino, la respuesta depende del tipo de sonda: - **Linux** (`traceroute`) usa por defecto **sondas UDP a puertos altos**; el destino responde **ICMP Destination Unreachable — Port Unreachable (tipo 3, código 3)**. - **Windows** (`tracert`) usa **ICMP Echo**; el destino responde **Echo Reply (tipo 0, código 0)**.

```
traceroute 16.0.5.130
tcpdump -i eth0 -w f-traceroute.pcap 'icmp or udp'
```

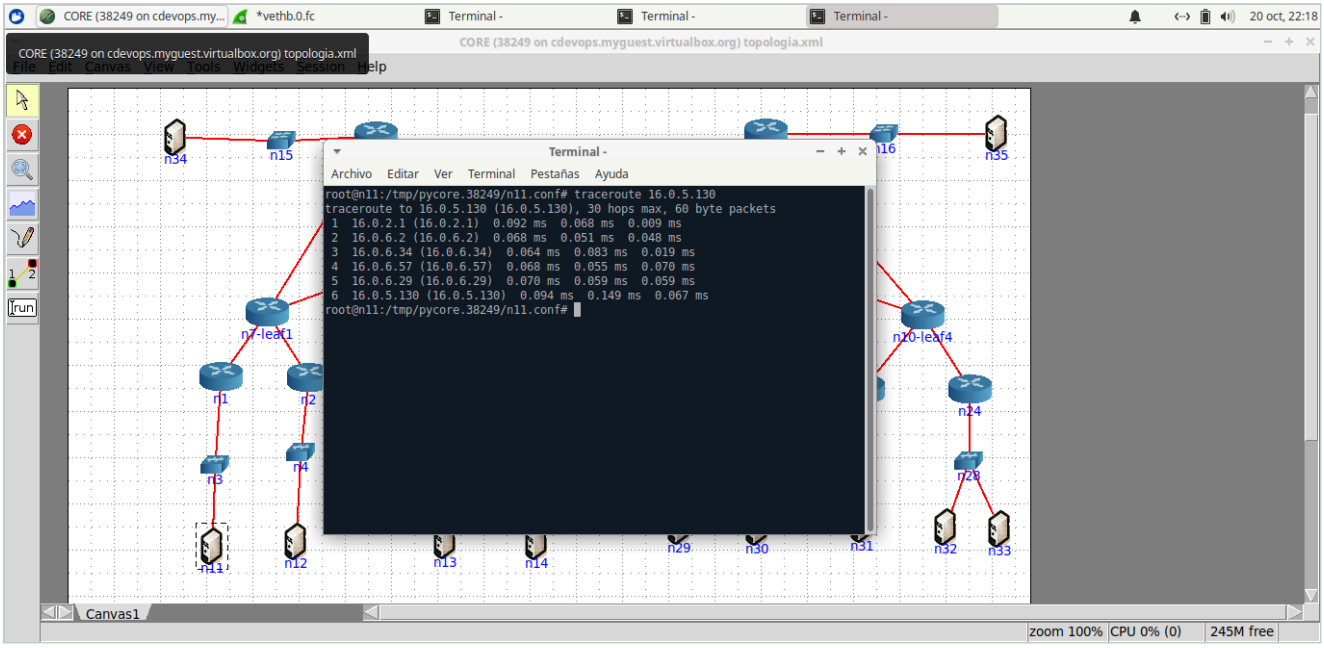


Figura F.1 — salida de `traceroute` de n11 a n33: los 6 saltos del camino

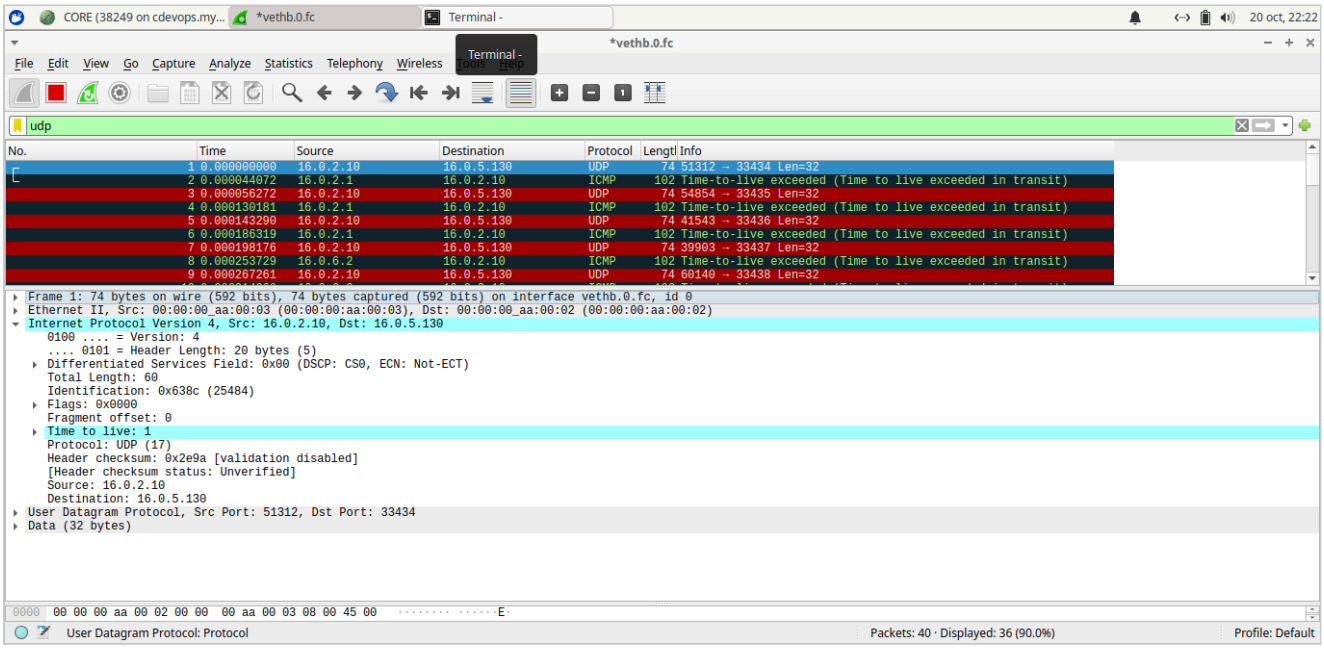
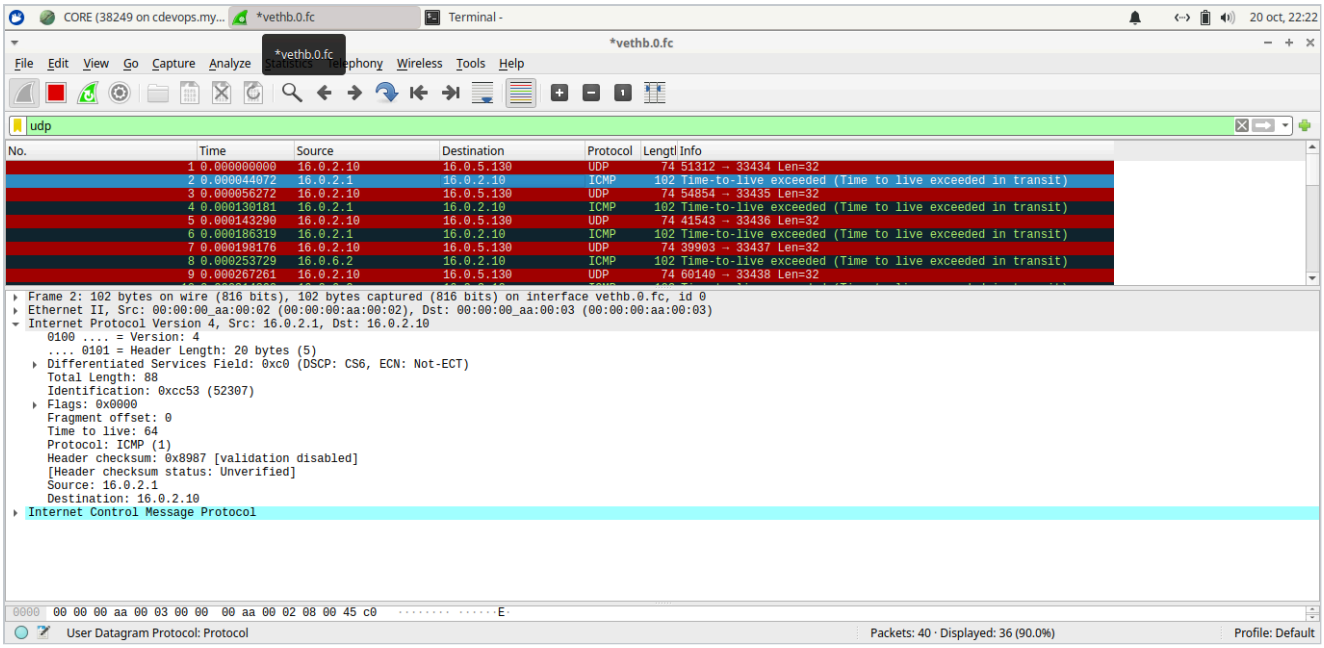
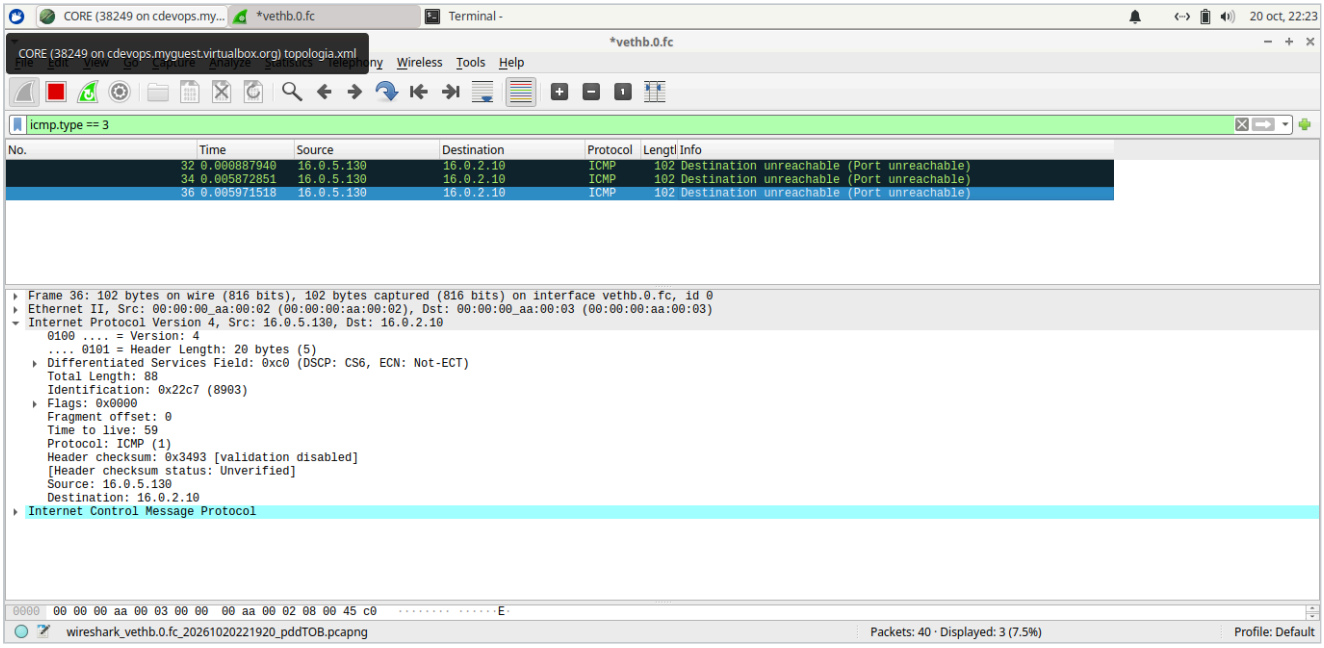


*Figura F.2 — primera sonda: UDP al puerto 33434 con TTL = 1**



> *Figura F.3 — ICMP Time Exceeded (11/0) desde `16.0.2.1`: el router se delata al descartar la sonda. El mensaje de error sale con TTL 64 propio, por ser un datagrama nuevo*



> *Figura F.4 — ICMP Port Unreachable (3/3) desde `16.0.5.130`: el cambio de tipo 11 a tipo 3 marca el fin del recorrido*

Inciso H — Conectividad IPv6 y análisis de ICMPv6

Verificación del túnel

La conectividad IPv6 entre Net5 y Net6 se verificó sobre el túnel 6in4, desde spine1:

```
ping6 -c 4 1600:0:0:ffff::2   # extremo remoto del túnel (spine2)
ping6 -c 4 1600:0:0:6::1      # LAN Net6, al otro lado del túnel
```

Ambas pruebas dieron **0% de pérdida**. El dato relevante es el **Hop Limit = 64 sin decrementar**: entre los dos spines hay un router IPv4 real (leaf1), pero para IPv6 el túnel constituye **un único salto**, porque leaf1 reenvía un datagrama IPv4 ordinario sin interpretar su contenido.

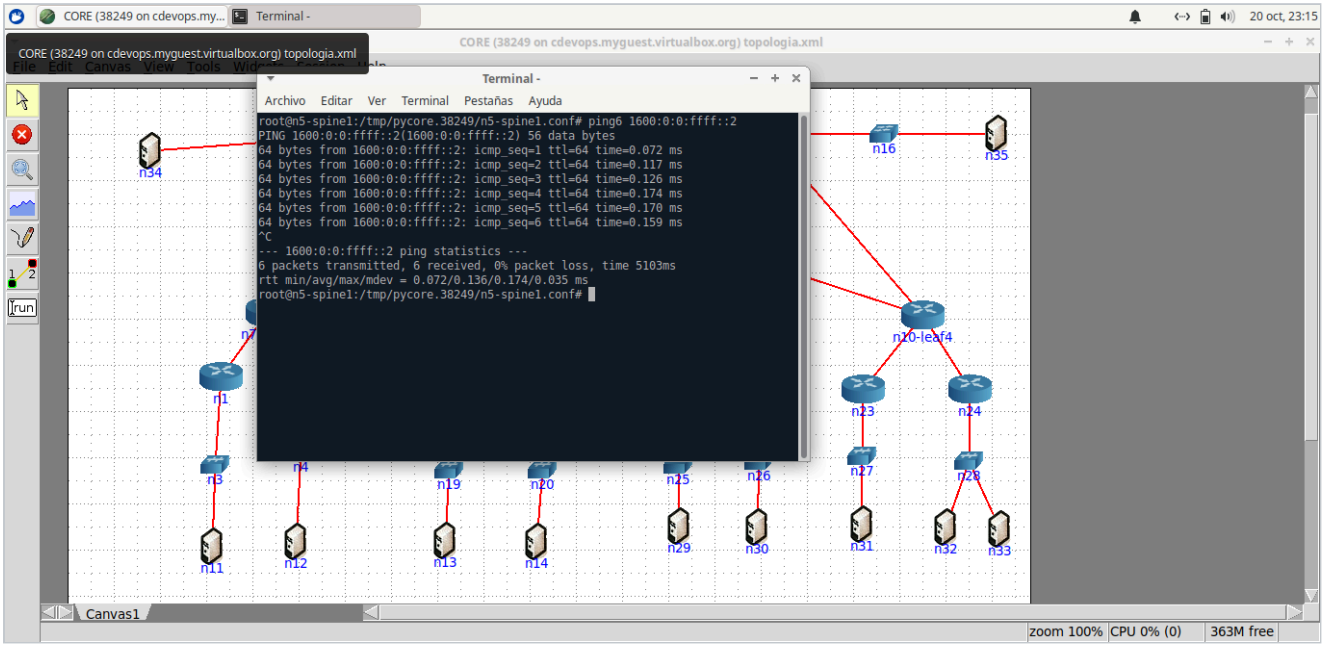


Figura H.1 — `ping6` de spine1 al extremo remoto del túnel (`1600:0:0:ffff::2`)

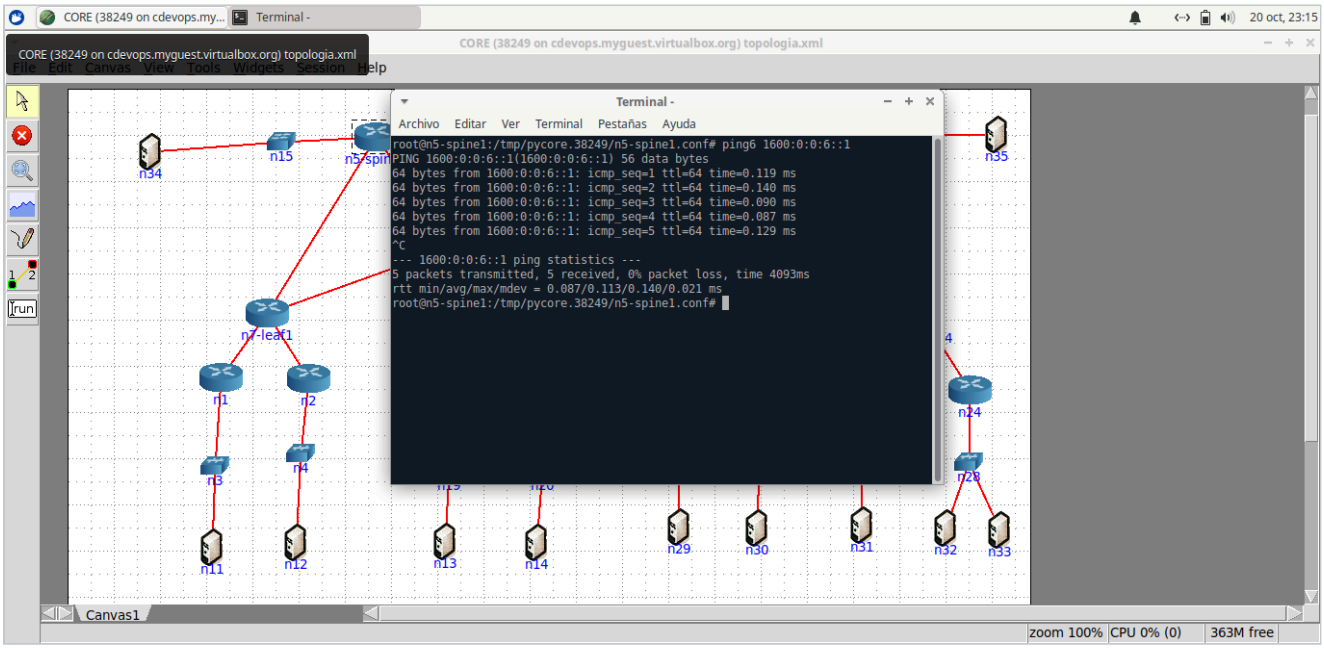
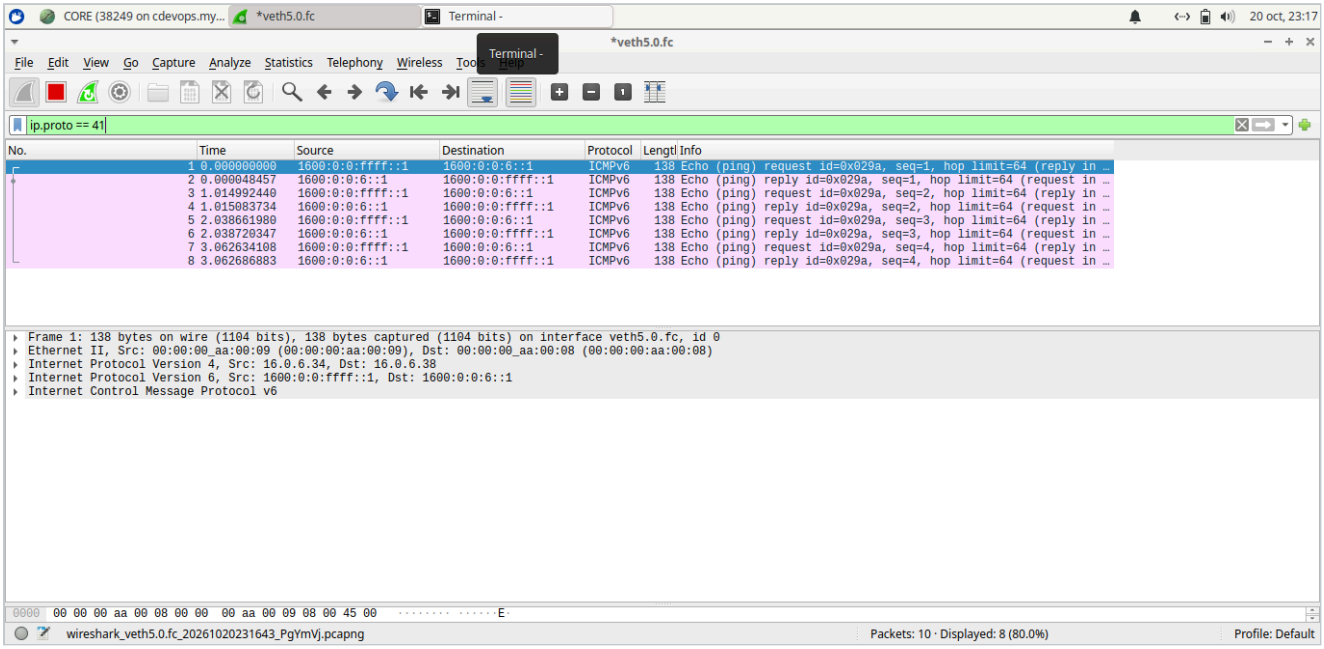


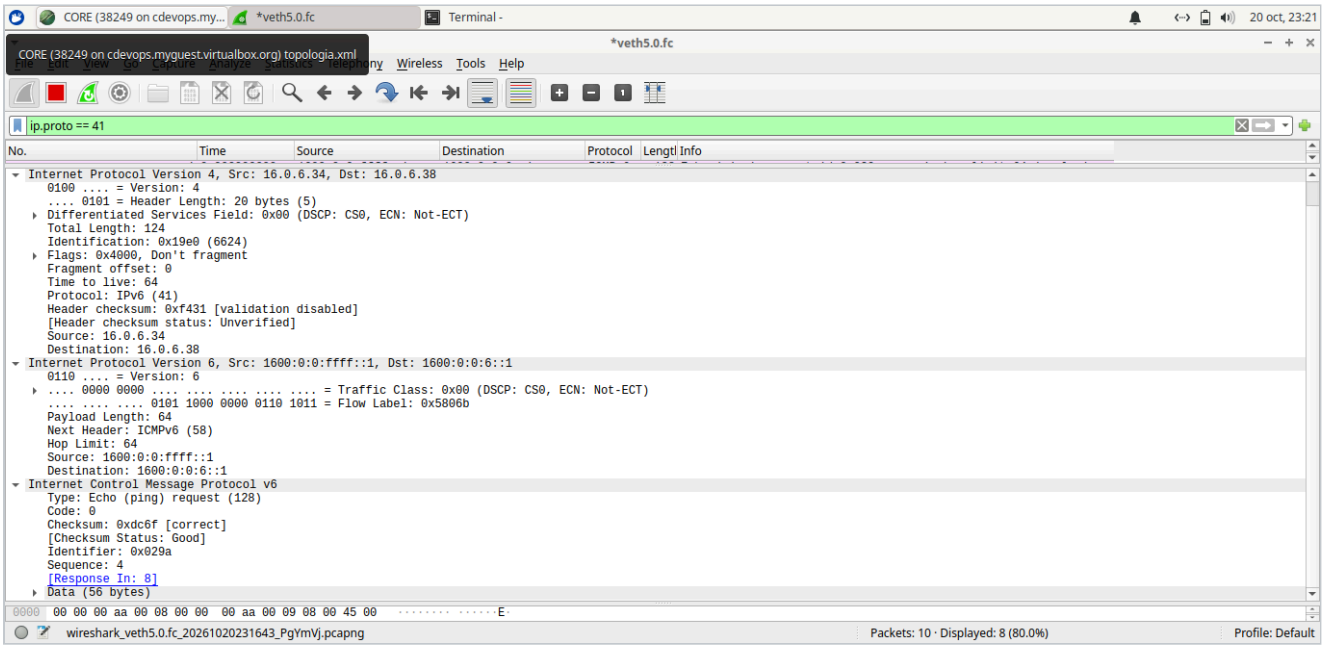
Figura H.2 — `ping6` de spine1 a la LAN Net6 (`1600:0:0:6::1`), atravesando el túnel

La encapsulación

Capturando en el enlace spine1 ↔ leaf1 con el filtro `ip.proto == 41` se observa la doble cabecera de red: un datagrama **IPv4** entre los extremos del túnel que transporta un datagrama **IPv6** completo.



> *Figura H.3 — el filtro es de IPv4 pero Wireshark muestra protocolo ICMPv6 y direcciones IPv6: las dos capas de red apiladas*



> *Figura H.4 — las tres cabeceras expandidas. IPv4 `Protocol: IPv6 (41)` de `16.0.6.34` a `16.0.6.38`; IPv6 `Next Header: ICMPv6 (58)` de `1600:0:0:ffff::1` a `1600:0:0:6::1`; ICMPv6 `Type: 128`*

El campo **`Protocol = 41`** de la cabecera IPv4 es el que identifica el encapsulamiento 6in4 (RFC 4213). Los tamaños lo confirman: `Total Length` IPv4 = 124 bytes = 20 (IPv4) + 40

(IPv6) + 64 (ICMPv6 y datos). Esos **20 bytes de sobrecarga** son los que reducen la MTU del túnel a **1480** (visible en la figura A.1).

Diferencias respecto de IPv4

	IPv4	IPv6
Resolución dirección ↔ MAC	**ARP**, protocolo independiente	**NDP**, dentro de ICMPv6
Mensajes	Request / Reply	**Neighbor Solicitation (135)** / **Neighbor Advertisement (136)**
Destino de la consulta	broadcast `ff:ff:ff:ff:ff:ff`	**multicast solicited-node** `ff02::1:ffXX:XXXX`
MAC destino	broadcast	`33:33:ff:` + últimos 3 bytes de la IPv6
Echo Request / Reply	8 / 0	**128 / 129**

El **Neighbor Solicitation** lleva **Hop Limit = 255**: un nodo descarta todo NDP cuyo Hop Limit sea distinto de 255, lo que garantiza que el mensaje no atravesó ningún router.

A diferencia del ARP Request, que interrumpe a todas las estaciones del segmento, el NS se envía a la dirección multicast *solicited-node*, formada con los últimos 24 bits de la dirección buscada: solo procesan la trama los nodos cuyo sufijo coincide.

Dos limitaciones del simulador

Durante la puesta en marcha aparecieron dos problemas que no corresponden al diseño sino al entorno, y que conviene dejar documentados.

1. Pérdida de la dirección link-local. El comando `ip addr flush dev <if>`, sin el modificador `-4`, elimina también la dirección **link-local** `fe80::` que el kernel genera al levantar la interfaz, y no la regenera mientras la interfaz siga activa. Sin link-local el NDP no opera, de modo que IPv4 funcionaba con normalidad e IPv6 quedaba inutilizable sobre el mismo enlace. Se corrigió el script para limpiar únicamente IPv4 y las IPv6 de *scope global*, preservando la link-local.

2. Filtrado del multicast NDP en los switches. Los switches del simulador son *bridges* de Linux, que pueden descartar el tráfico multicast IPv6. Con `tcpdump` se comprobó que el Neighbor Solicitation **salía** de n34 hacia `ff02::1:ff00:1` pero **no llegaba** a la interfaz de spine1:

```
n34    → IP6 1600:0:0:5::10 > ff02::1:ff00:1: neighbor solicitation, who has 1600:0:0:5::1
spine1 → 0 packets captured
```

Esto explica con precisión por qué ARP funcionaba e NDP no: **ARP usa broadcast (`ff:ff:ff:ff:ff:ff`), que el bridge siempre inunda, mientras que NDP usa multicast (`33:33:ff:..`), que queda sujeto a filtrado.** La resolución de vecinos se cargó entonces de forma estática:

```
ip -6 neigh replace 1600:0:0:5::1  lladdr 00:00:00:aa:00:0a dev eth0   # en n34
ip -6 neigh replace 1600:0:0:5::10 lladdr 00:00:00:aa:00:0b dev eth1   # en spine1
```

Por esa limitación la verificación se realizó **entre los extremos del túnel**, que es donde reside el mecanismo de transición objeto del inciso y donde no interviene ningún bridge. El direccionamiento y el ruteo IPv6 de las LAN son los indicados en las figuras A.1, A.2, B.15 y B.16.

Inciso C — NAT con direccionamiento privado *(alternativo)*

Se asigna a **n33** una dirección privada según **RFC 1918**, que define tres rangos:

```
10.0.0.0    – 10.255.255.255     (1 red clase A)
172.16.0.0  – 172.31.255.255     (16 redes clase B)
192.168.0.0 – 192.168.255.255    (256 redes clase C)
```

Se elige `192.168.0.0/24`, con `192.168.0.10` para n33 y `192.168.0.1` para la interfaz de n24 que oficia de gateway. En n24 la dirección privada se **agrega** a la que ya tenía, de modo que n32 conserva su conectividad por `16.0.5.0/27`; en n33, en cambio, se **reemplaza**: queda únicamente con la privada.

```
# n24
ip addr add 192.168.0.1/24 dev eth0
sysctl -w net.ipv4.ip_forward=1
iptables -P FORWARD ACCEPT
iptables -t nat -A POSTROUTING -s 192.168.0.0/24 -o eth1 -j MASQUERADE

# n33
ip addr del 16.0.5.130/27 dev eth0
ip addr add 192.168.0.10/24 dev eth0
ip route replace default via 192.168.0.1
```

Se usa **MASQUERADE** (NAPT) y no SNAT estático porque traduce contra la dirección que tenga la interfaz de salida sin necesidad de fijarla en la regla.

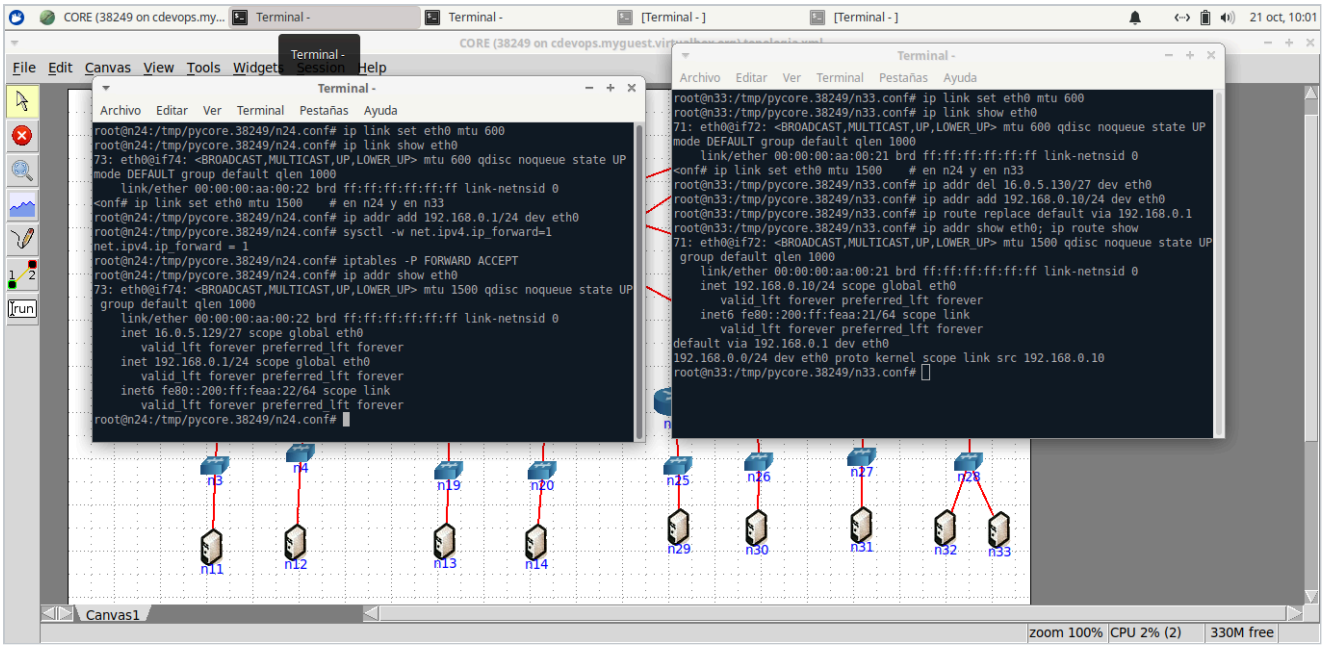


Figura C.1 — n24 con ambas direcciones y n33 únicamente con `192.168.0.10/24` y su ruta por defecto

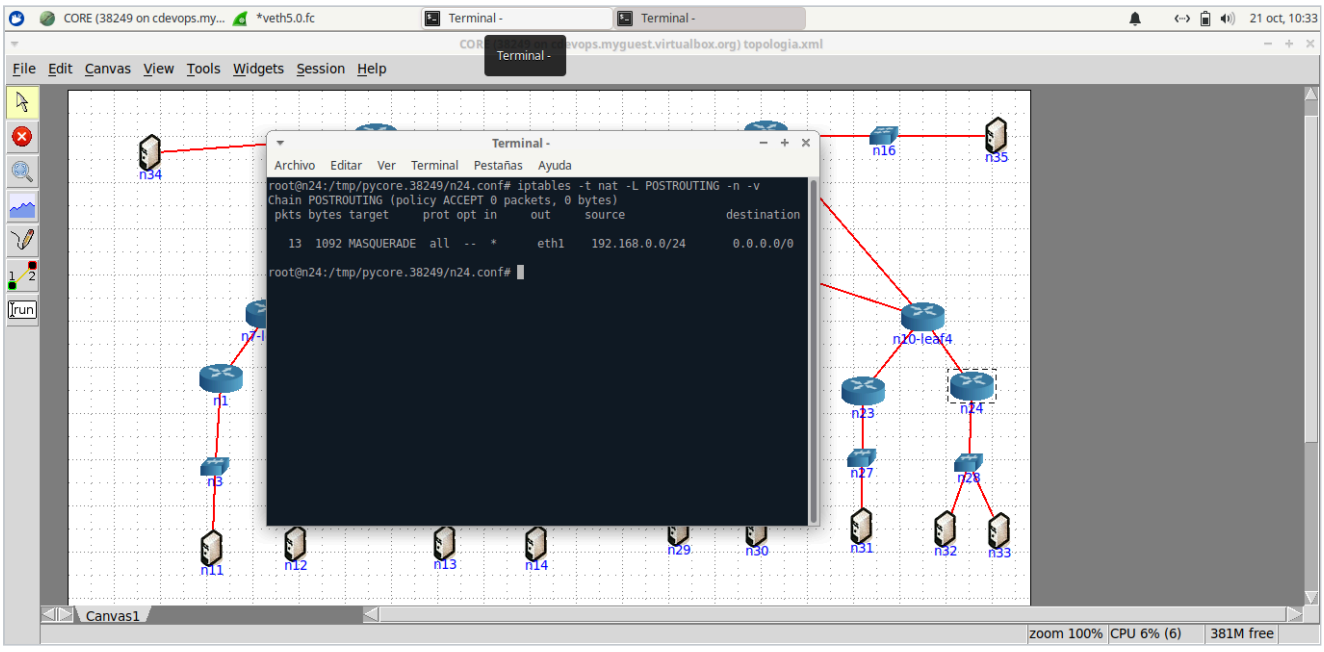


Figura C.2 — la regla MASQUERADE con sus contadores: 13 flujos traducidos

El contador registra **conexiones, no paquetes**: la tabla `nat` se consulta solo para el primer paquete de cada flujo y los siguientes se traducen a partir de la entrada ya creada en `conntrack`. Por eso un `ping -c 60`, que es un único flujo ICMP, lo incrementa en 1.

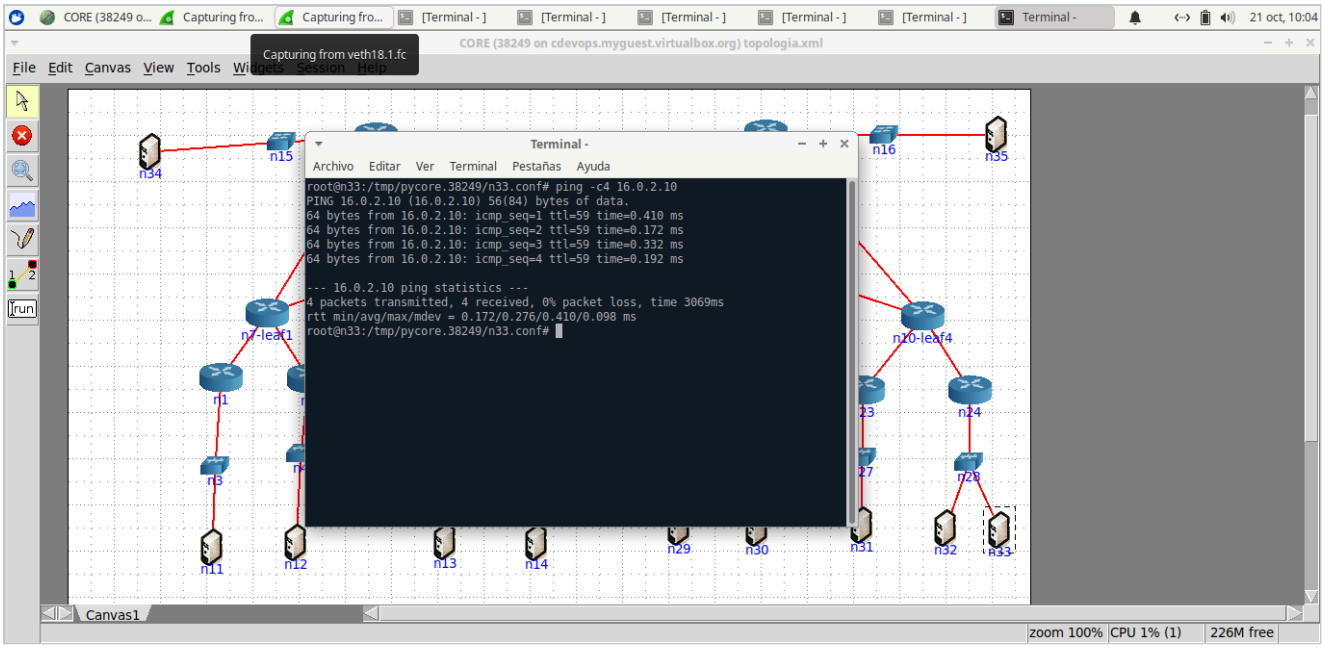


Figura C.3 — n33, con dirección privada, alcanza n11 al otro extremo de la topología

La traducción, capturada a ambos lados de n24

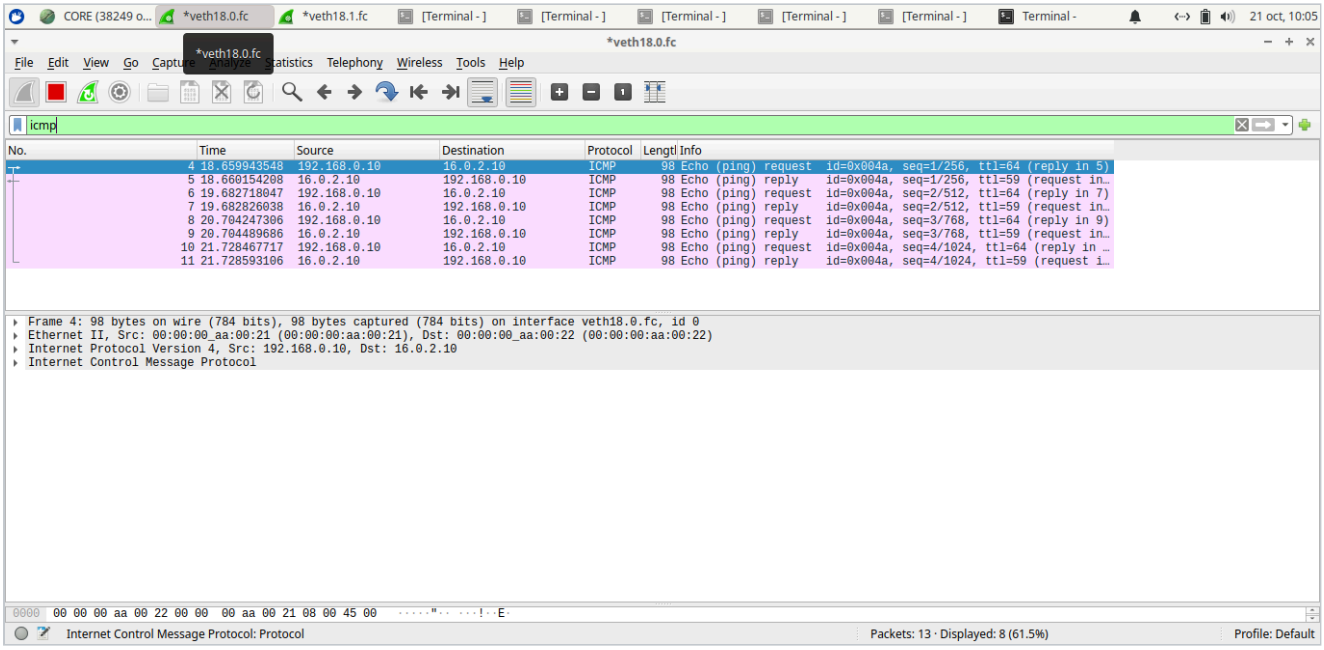
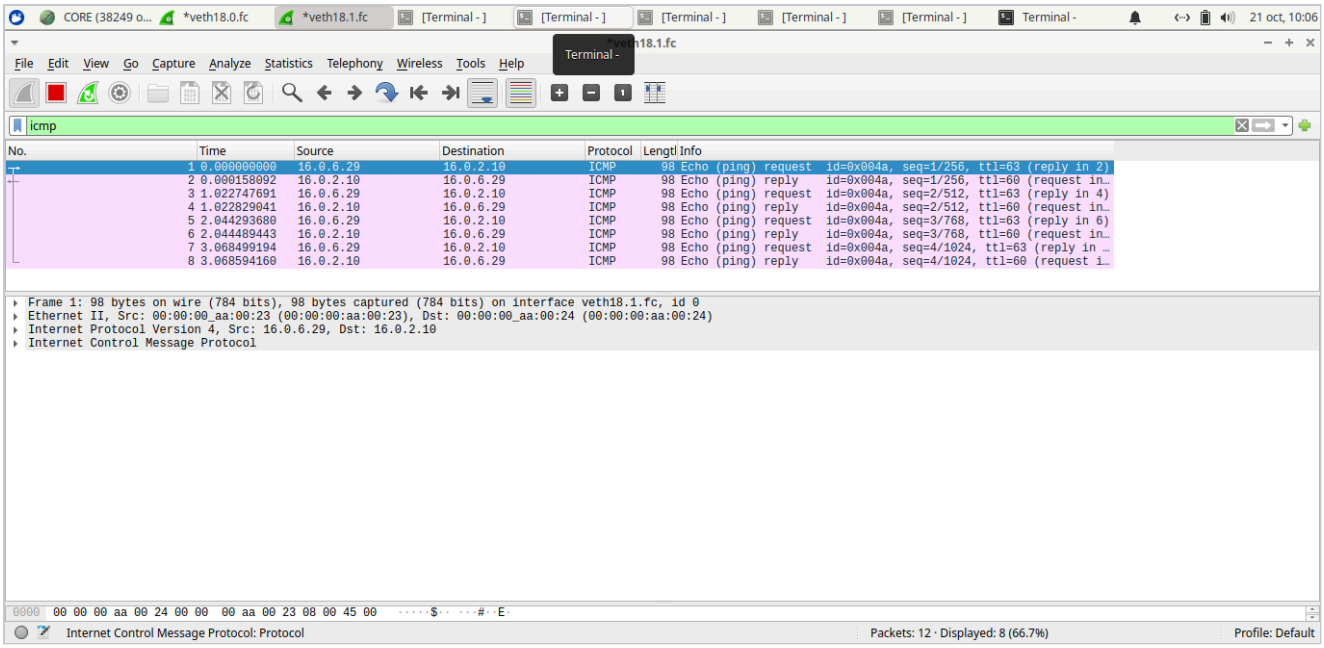


Figura C.4 — lado privado (n24 ↔ n28): origen `192.168.0.10`



> *Figura C.5 — lado público (n24 ↔ leaf4): origen `16.0.6.29`, con el mismo `id` y `seq`*

	Lado privado	Lado público
Source	`192.168.0.10`	**`16.0.6.29`**
Destination	`16.0.2.10`	`16.0.2.10`
ICMP `id`	`0x004a`	**`0x004a`**
`seq`	1/256	**1/256**
TTL	64	**63**

El **`id` y el `seq` idénticos** prueban que se trata del mismo paquete con la dirección de origen reescrita; el **TTL decrementado** confirma que n24 lo ruteó. Como el tráfico sale con origen `16.0.6.29` —una dirección de enlace punto a punto—, el retorno depende de las rutas de tránsito `/29` sumarizadas que se agregaron a los spines en el inciso b): **sin ellas el NAT no funcionaría**.

Inciso G — Modificación de MTU y fragmentación *(alternativo)*

IPv4: fragmenta el router

Se reduce a **600 bytes** el MTU del enlace entre n24 y n33, en ambos extremos, y se envía desde n11 un datagrama que no entra en él.

```
ip link set eth0 mtu 600          # en n24 y en n33
ping -c2 -s 1400 -M dont 16.0.5.130    # desde n11
```

`-M dont` desactiva el bit *Don't Fragment*: sin él el router descartaría el datagrama en lugar de partirlo. El tamaño total es `1400 + 8 (ICMP) + 20 (IP) = 1428 bytes`.

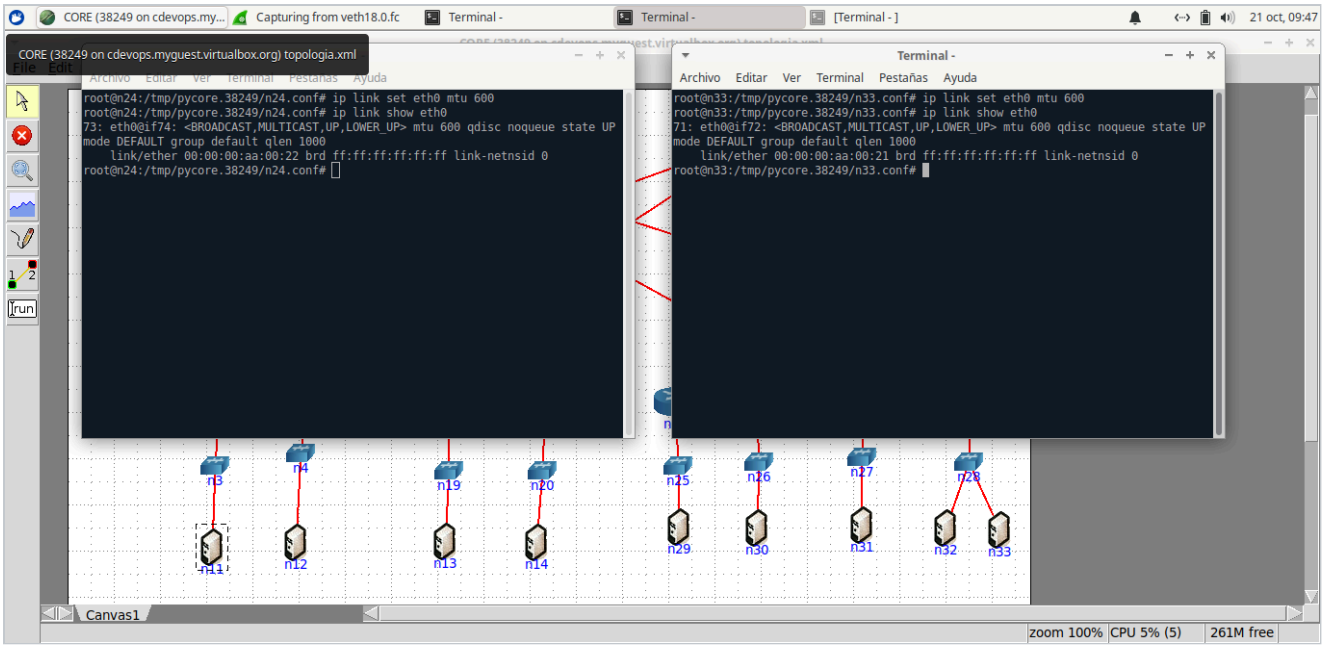


Figura G.1 — MTU reducido a 600 en los dos extremos del segmento de n33

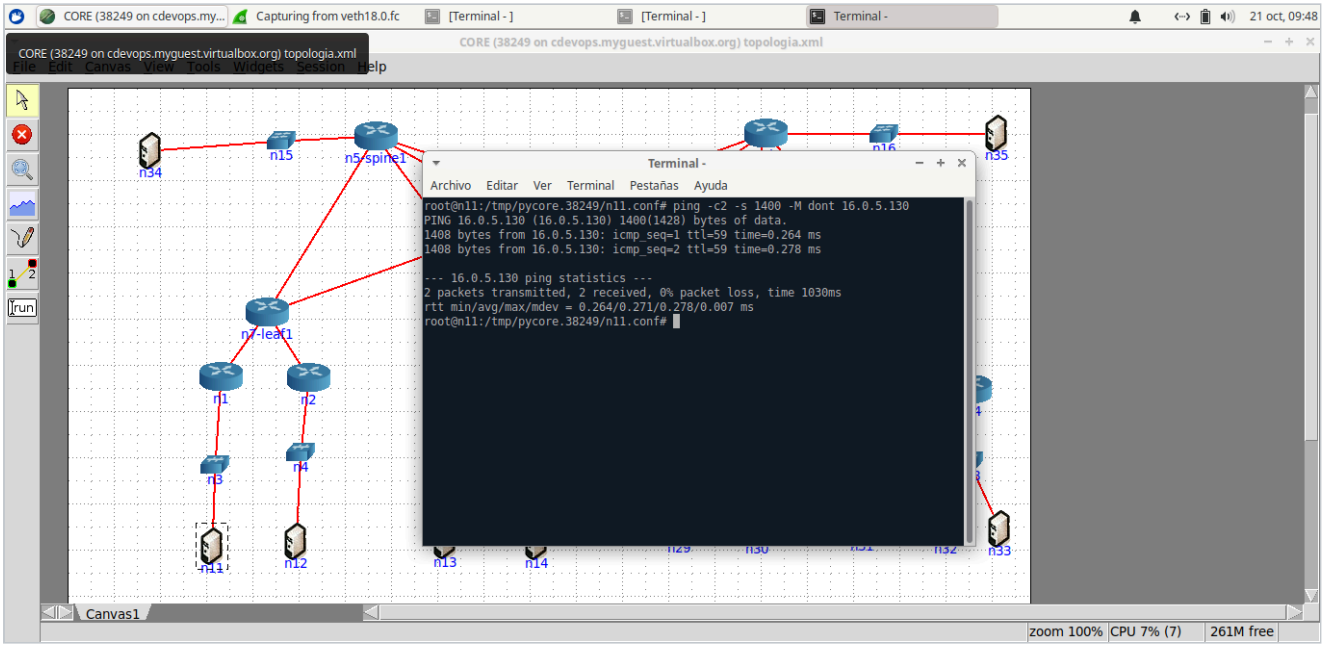
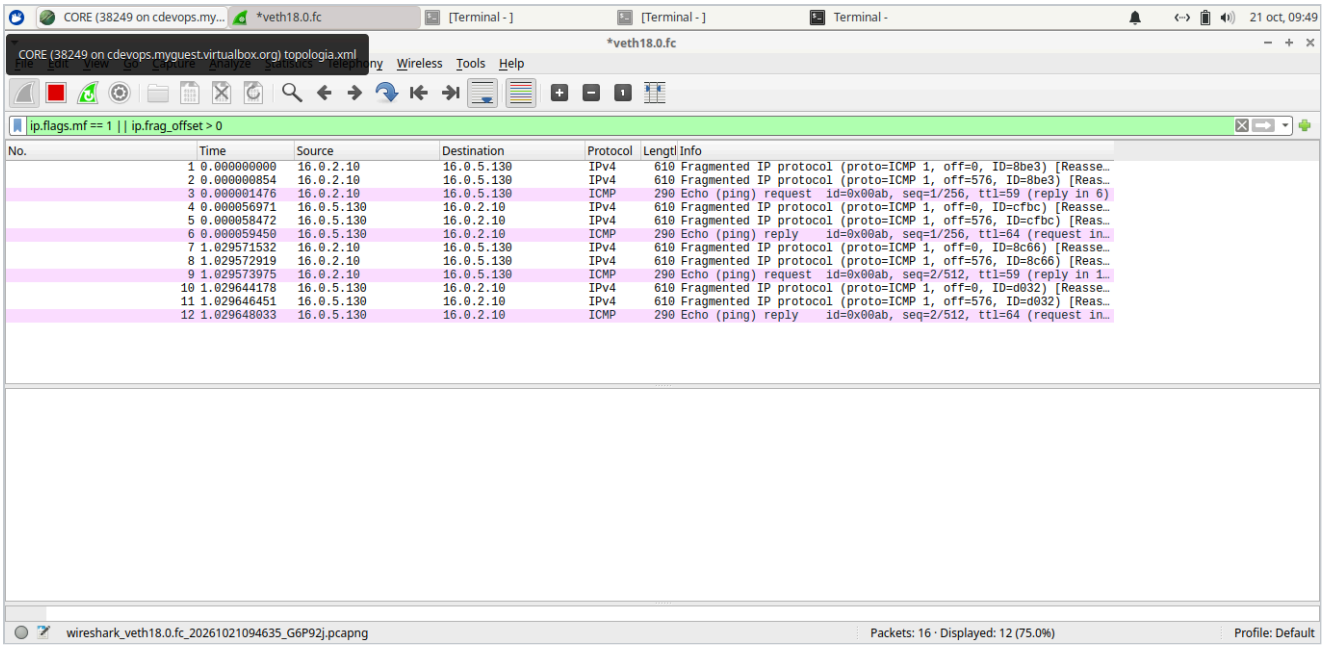
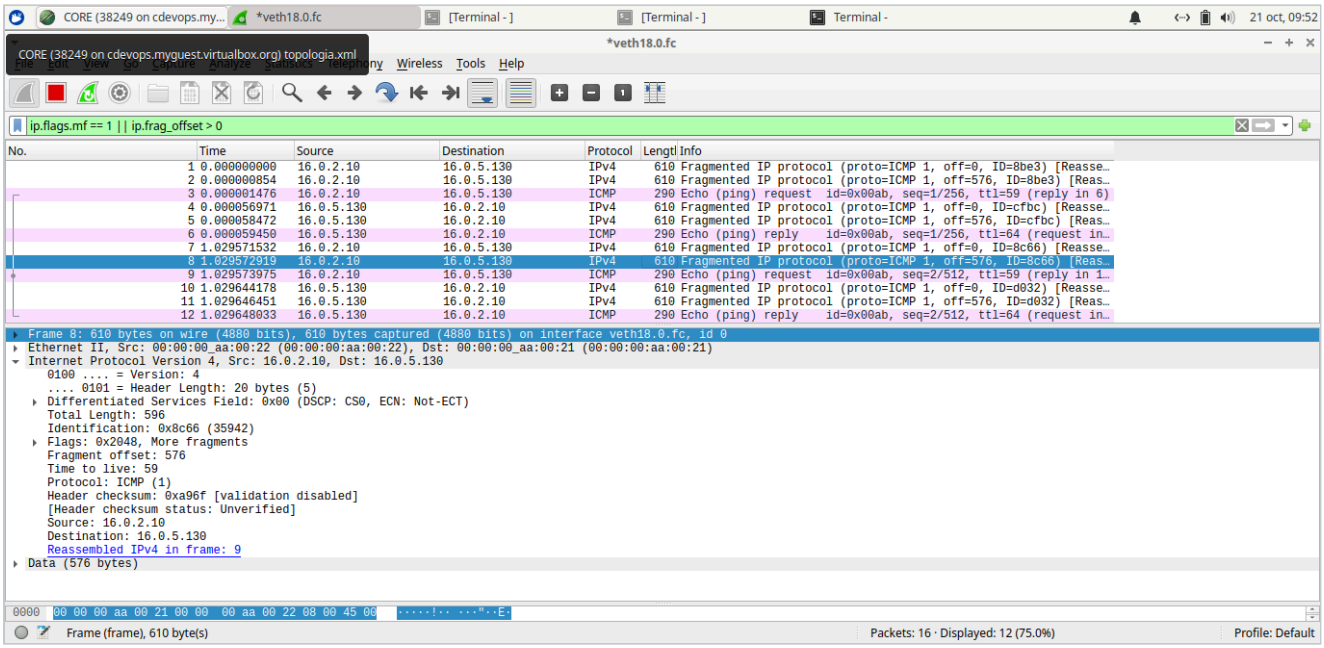


Figura G.2 — el ping responde con 0% de pérdida: n24 fragmentó y n33 reensambló



> *Figura G.3 — los tres fragmentos por datagrama, con el filtro `ip.flags.mf == 1 || ip.frag_offset > 0`*

Los tamaños se desprenden del MTU: `600 − 20 = 580`, redondeado al múltiplo de 8 inferior da **576 bytes** de datos por fragmento, y `1408 − 576 − 576 = 256` en el último. De ahí los datagramas de **596 / 596 / 276 bytes**.



> *Figura G.4 — el fragmento intermedio: `Identification` compartido, `More fragments` activo y `Fragment offset` distinto de cero*

Los tres campos que permiten el reensamblado:

Campo	Valor	Función
`Identification`	`0x8c66` , **igual en los tres**	agrupa los fragmentos de un mismo datagrama
`More fragments`	1, 1, **0**	marca cuál es el último
`Fragment offset`	0, **72**, 144	ubica cada trozo dentro del original

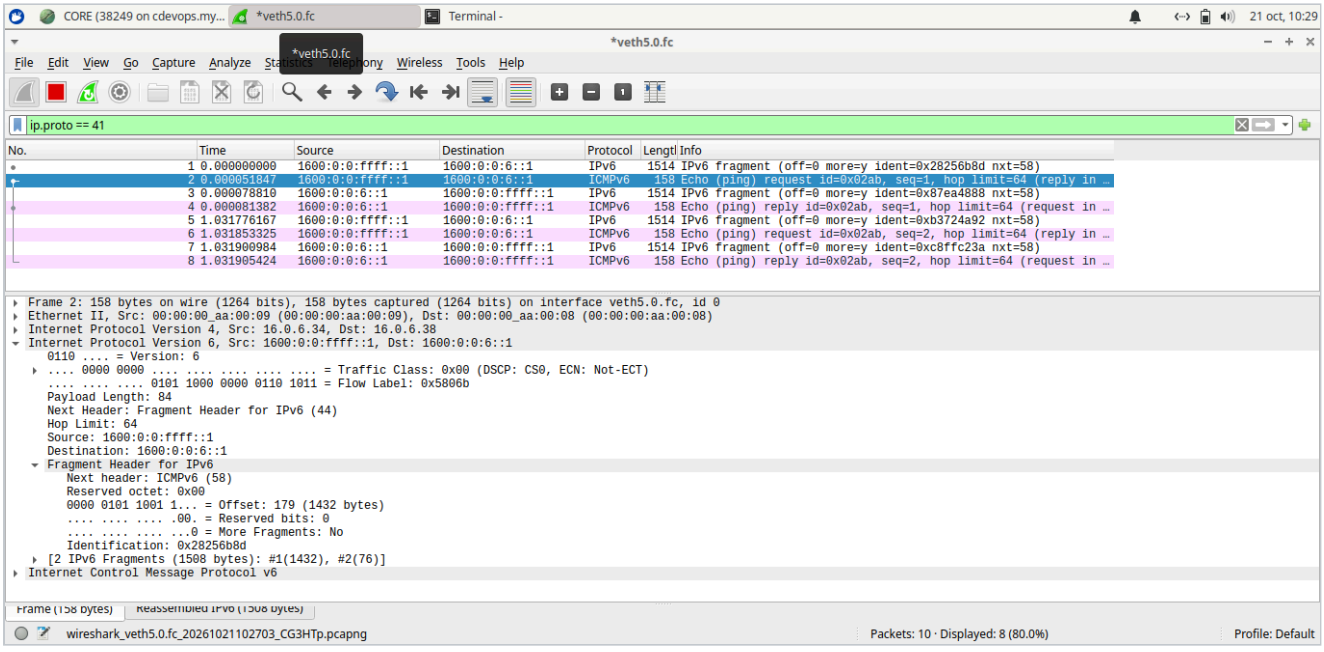
> El campo *Fragment Offset* se expresa en **unidades de 8 bytes**: el valor almacenado es 72, aunque Wireshark lo presente ya multiplicado, como 576 bytes.

IPv6: fragmenta el origen

En IPv6 **los routers intermedios no fragmentan**: si el datagrama no entra en el enlace siguiente, el router lo descarta y devuelve **ICMPv6 Packet Too Big (tipo 2)**, y es el nodo origen quien debe reducir el tamaño. La fragmentación, cuando ocurre, se señaliza con una **cabecera de extensión Fragment (Next Header 44)** en lugar de campos de la cabecera fija.

No hizo falta modificar ningún MTU: **el túnel 6in4 ya tiene MTU 1480**, por los 20 bytes de la cabecera IPv4 externa.

```
ping6 -c2 -s 1500 -M dont 1600:0:0:6::1    # desde spine1
```



> *Figura G.5 — fragmentación IPv6 dentro del túnel:* `Next Header: Fragment Header for IPv6 (44)` *, offset 179 (1432 bytes) y el reensamblado de 2 fragmentos (1432 + 76 = 1508)*

Acá quien fragmenta es **spine1, el origen**, no un router del camino. Esa es la diferencia de fondo con el caso IPv4, donde el que partió el datagrama fue n24, un router intermedio.

	IPv4	IPv6
¿Quién fragmenta?	origen **y routers intermedios**	**solo el origen**
¿Dónde van los campos?	en la cabecera fija	en una **cabecera de extensión (44)**
Si no entra en un router	el router lo parte	**ICMPv6 Packet Too Big (tipo 2)**
MTU mínima	68 bytes	**1280 bytes**

Conclusiones

La topología spine-leaf quedó operativa con **ruteo exclusivamente estático** y conectividad verificada extremo a extremo. Del trabajo se desprenden cuatro observaciones.

El VLSM asignado de mayor a menor evita la fragmentación del espacio. Las diez redes consumen 1520 de las 65 536 direcciones del bloque (2,3%), cada bloque alineado a su propio tamaño y sin solapamientos, dejando el resto disponible para crecer.

La disciplina de ruteo es lo que vuelve imposible el bucle. Rutas por defecto únicamente hacia arriba, rutas específicas hacia abajo y **spines sin ruta por defecto**. La figura D.3 lo demuestra: el ping a una red inexistente devuelve *Net Unreachable* desde el propio spine, y no *Time Exceeded*, que sería el síntoma de un lazo.

Las redes de tránsito también deben ser alcanzables. El primer diagnóstico de falla —ping fallido entre n1 y n24, pero exitoso forzando la IP de origen de la LAN— se debía a que los spines conocían las LAN pero no las `/30` de los enlaces punto a punto, por lo que descartaban el tráfico de retorno. Como los dos enlaces de cada leaf quedaron contiguos y alineados, se resolvió con **ocho rutas `/29` sumarizadas** en lugar de dieciséis `/30`.

El túnel 6in4 confina la complejidad IPv6 a dos nodos. Los cuatro leafs operan únicamente IPv4 y no requieren configuración alguna de IPv6. El costo es de 20 bytes por datagrama y una MTU de 1480.